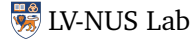




Scaling Harness Intelligence via *Just-in-Time* Harness Evolution



Abstract

Agent capability is not determined by the model alone. The agent harness, encompassing memory management, planning strategy, action protocol, and tool/skill orchestration, can dominate the contribution of the underlying foundation model. Yet harness design remains manual, task-specific, and fundamentally unscalable. We present JIT-Agent, a harness intelligence model trained to synthesize task-adaptive agent harnesses on the fly for arbitrary off-the-shelf agentic LLMs. We formalize the agent harness as a composable, machine-generatable artifact governed by a fixed four-module protocol, and train JIT-Agent to *customize harnesses* for a given task at hand, *repair harnesses* for stable and reliable execution, and *self-evolve* by distilling performance signals from an expanding archive of prior harness configurations. Equipped with JIT-Agent as a harness helper, DeepSeek-V4-Flash surpasses GPT-5.6 on DeepSearchQA (+9.1) and OdysseyBench (+4.3), while the already strong GLM-5.2 gains up to +20.2 points. Across controlled evaluations, JIT-Agent-generated harnesses are performance-competitive with mature agent runtimes such as OpenCode and Claude Code and consistently improve multi-scale model families of DeepSeek V4, MIMO-V2.5, and Qwen3.6. To our knowledge, JIT-Agent is the first model purpose-built for just-in-time harness generation, establishing harness intelligence as a trainable, transferable, and compounding dimension of agent capability orthogonal to model scaling.

Website <https://bingreeky.github.io/JIT-site>
GitHub <https://github.com/bingreeky/JIT>
Hugging Face <https://huggingface.co/JIT-Agent>

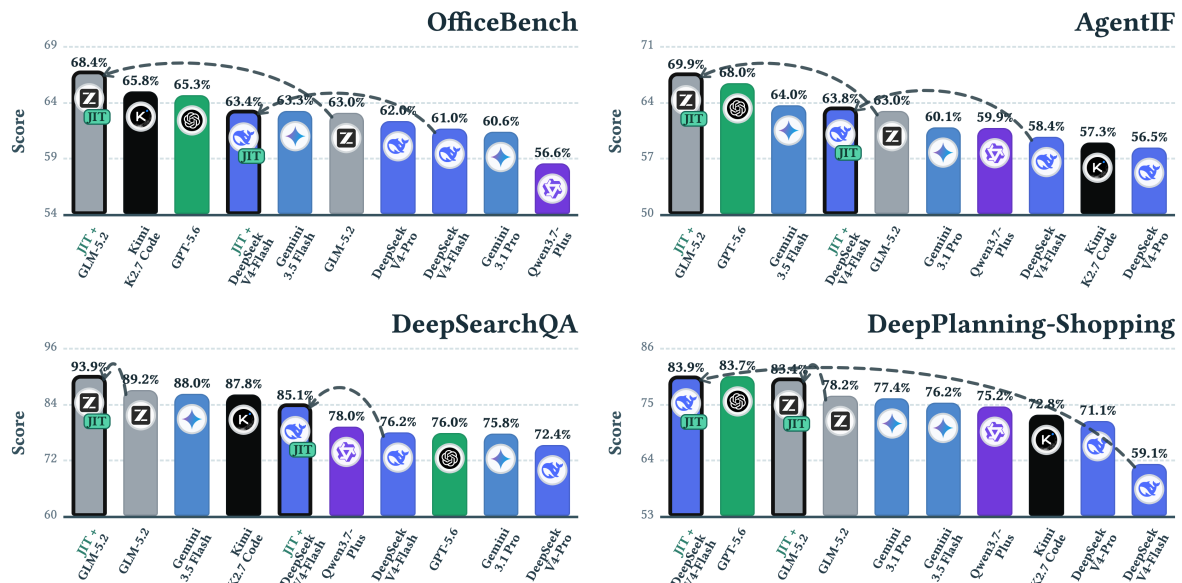


Figure 1: Leaderboard across four representative agent benchmarks. JIT-Agent-generated harnesses consistently improve the underlying backbone agents across deep research, daily work, planning, and workspace tasks, demonstrating that just-in-time harness synthesis provides gains beyond model scaling alone.

1 Introduction

Agent harness matters. The capability of an LLM agent is jointly determined by two tightly coupled factors: the *foundation model* that produces reasoning and actions, and the *agent harness* that situates this model in a closed-loop execution environment (Meng et al., 2026b; Zhou et al., 2026; Ning et al., 2026). The harness decides what history is retained, how local intent is formed, which tools or skills are exposed, how actions are executed, and when verification or recovery is triggered (Meng et al., 2026b; Zhou et al., 2026). A strong model may fail when placed behind the wrong memory, planner, or action protocol; a strong harness, in turn, can only unlock capability when the model can understand and follow it (Lee et al., 2026c; Lin et al., 2026). Thus, agent intelligence is not a property of model weights alone, but of the model-harness pair (Lee et al., 2026c; Zhou et al., 2026). Harness design is therefore not an implementation detail, but a first-order determinant of agent performance (Meng et al., 2026b; Lin et al., 2026).

Ahead-of-Time (AOT) Harness This observation has motivated a growing line of work on test-time harness optimization (Lee et al., 2026c; Lin et al., 2026). Recent systems optimize harness code, prompts, tools, memories, skills, or control policies from trajectories and feedback, yielding strong improvements in coding, terminal-use, web interaction, and general agentic domains (Lee et al., 2026c; Lin et al., 2026). Despite their differences, many of these methods share an *Ahead-of-Time* (AOT) assumption: the harness is treated as a durable artifact to be optimized over an experience stream, with the hope that the resulting artifact will generalize across future tasks, domains, or model versions (Lee et al., 2026c; Lin et al., 2026). This is a powerful paradigm when the deployment distribution is stable and homogeneous (Lin et al., 2026; Zhang et al., 2025b). However, it still asks the optimization loop to precompile a broadly useful harness before seeing the exact structure of each future problem (Lee et al., 2026c).

Just-in-Time (JIT) Harness. We instead turn to a *Just-in-Time* (JIT) view of harness construction. Different tasks plainly require different harness priors. Wide-search tasks may benefit from parallel evidence exploration (Qin et al., 2025); terminal tasks often favor a lean serial ReAct loop (Yao et al., 2023a; Merrill et al., 2026); deep-research tasks require working memory over retrieved evidence (Hu et al., 2025); and NL2Repo-style coding tasks are naturally mediated by a filesystem that stores patches, tests, traces, and repository state (Yang et al., 2024; Ning et al., 2026). In short, the appropriate harness is not only domain-dependent, but instance-dependent (Meng et al., 2026b; Zhou et al., 2026). Optimizing a single AOT harness across such heterogeneous demands is cumbersome: one must search a large design space, accumulate enough trajectories, and then hope that the resulting scaffold matches the next task (Lee et al., 2026c; Lin et al., 2026). We ask whether another possibility is viable: **Model-as-a-Harness**, where a trained meta-agent generates a task-specific harness on the spot, and an arbitrary agentic LLM executes under that harness.

JIT-Agent. We propose JIT-Agent, a compact meta-agent for just-in-time harness generation and evolution.¹ At inference time, JIT-Agent receives a task specification, a protocol, an executable tool/skill registry, and a small retrieved context of prior harnesses, and emits an executable harness tailored to the task at hand. The resulting harness wraps an off-the-shelf agentic LLM for execution. As feedback and traces accumulate, JIT-Agent can revise the harness and update a harness archive, enabling test-time harness evolution while the generator itself remains fixed. Training such a system raises three challenges: ❶ **adaptivity**, namely how to match the generated harness to the task; ❷ **reliability**, namely how to ensure executable behavior and recover when synthesis fails (Pan et al., 2026a; Ning et al., 2026); and ❸ **evolvability**, namely how to turn execution feedback into stronger

¹JIT-Agent continues our broader effort toward the recursive self-design and self-evolution of agent systems. Before addressing the harness as a whole, we studied modular design and evolution at the level of individual harness components: memory in MEMEVOLVE (Zhang et al., 2025b) and planning in TODOEVOLVE (Liu et al., 2026a). JIT-Agent is the unified continuation of this progression, extending component-level self-design to the full operational harness.

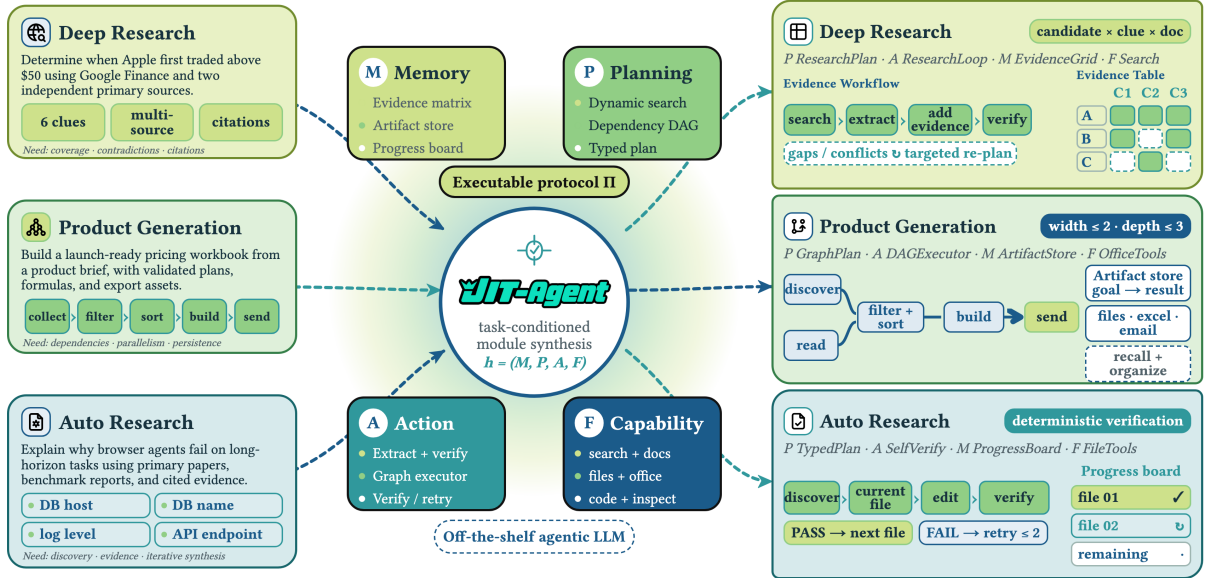


Figure 2: Overview of JIT-Agent. Given a task, JIT-Agent composes a problem-specific agent harness by instantiating (rather than simply combining) four modules: memory, planning, action, and capability. Different task structures therefore induce distinct executable protocols and state organizations, as illustrated by the specialized harnesses for deep research, product generation, and autonomous research.

future harnesses (Lee et al., 2026c; Lin et al., 2026). Together, these three requirements define what we call **harness intelligence**:

Harness Intelligence

Harness intelligence is the capacity to construct and refine the operational scaffold through which a model acts. It has three defining properties: **adaptivity**, matching the harness to the task and backbone; **reliability**, producing executable behavior and recovering when synthesis fails; and **evolvability**, turning execution feedback into stronger future harnesses. Intelligence about the harness begins where competence within a fixed harness ends.

JIT-Agent realizes harness intelligence through a three-stage training recipe over a protocol-induced harness space (Meng et al., 2026b; Zhou et al., 2026). First, we factor an agent harness into a four-tuple of *memory*, *planning*, *action*, and *capability-orchestration* modules, and build HARNESSFACTORY, a unified codebase of representative scaffolds under this common interface (Hu et al., 2025; Erdogan et al., 2025a; Yao et al., 2023a; Xu and Yan, 2026). Following the view that *code carries the harness* (Ning et al., 2026; Pan et al., 2026a), this representation makes harness generation feasible: JIT-Agent generates structured executable modules rather than unconstrained agent programs. **① Stage I** then teaches task-conditioned customization from teacher-generated protocol-compliant examples, giving the generator adaptivity over the shared design space. Second, **② Stage II** converts failed generations into bounded repair trajectories, turning compiler errors, interface mismatches, and runtime failures into supervision for reliable recovery. Third, **③ Stage III** introduces *Evolutionary Group-Decoupled Policy Optimization* (Evo-GDPO), which optimizes JIT-Agent to propose harnesses that overtake the current archive frontier while separately normalizing reward, latency, and cost. This supplies evolvability by turning test-time harness improvement into a trained capability rather than an external search heuristic. Through this pipeline, we train **JIT-Agent-27B**, a **harness intelligence model** that can customize, repair, and evolve task-specific harnesses, shifting harness engineering from AOT artifacts to native JIT synthesis.

Experiment Findings. The results show that harness intelligence can both elevate an efficient backbone and extend an already capable one. DeepSeek-V4-Flash equipped with JIT-Agent surpasses

GPT-5.6 on DeepSearchQA (+9.1), PinchBench (+8.7), and OdysseyBench (+4.3). On already strong GLM-5.2, JIT-Agent raises the ceiling further, with large absolute gains on xBench-DS (+12.0) and AgentIF (+6.9). Beyond these headline results, JIT-Agent-generated harnesses are performance-competitive with mature agent runtimes such as OpenCode and Claude Code and deliver consistent gains across differently scaled variants of DeepSeek V4, MIMO-V2.5, and Qwen3.6.

Contributions. Our contributions are summarized as follows:

- **Harness Intelligence.** We formulate *Model-as-a-Harness* as a new agent-improvement paradigm and define *harness intelligence* as the learned ability to construct, repair, and evolve an agent’s operational scaffold.
- **Practical Solution.** We introduce JIT-Agent and its three-stage training pipeline, combining a four-module executable harness protocol, HARNESSFACTORY, customization learning, feedback-driven repair, and Evo-GDPO for test-time harness evolution.
- **Experimental Validation.** We validate JIT-Agent across heterogeneous agentic benchmarks and model backbones, showing that JIT-generated harnesses improve vanilla agents, compete with strong fixed harnesses, and move backbone–harness pairs toward better cost–performance frontiers.

2 Related Work

Harness Engineering and Modularization. Harness engineering remains strikingly diverse, spanning coding and general-purpose runtimes such as Claude Code, Codex, Hermes, and OpenClaw (Anthropics, 2025; OpenAI, 2025; Nous Research, 2026; OpenClaw Contributors, 2026), embodied systems (Zhang et al., 2026e; Lee et al., 2026b), search agents (Apodex Team, 2026; Qin et al., 2025), and memory-centric harnesses such as Mem0, EverMemOS, and MemEvolve (Chhikara et al., 2025; Hu et al., 2026a; Zhang et al., 2025b). Beneath this variety, the harness is the operational layer that determines what an agent observes, remembers, plans, invokes, verifies, and executes, making capability a property of the model–harness pair rather than of model weights alone (Meng et al., 2026b; Zhou et al., 2026; Pan et al., 2026a; Lee et al., 2026c). Recurring design choices span (i) **memory**, which curates working context and preserves cross-session experience (Hu et al., 2026b, 2024a; Yan et al., 2025; Zhang et al., 2025c); (ii) **planning**, which forms and revises subgoals to update the agent’s direction (Cao et al., 2025; Erdogan et al., 2025b; Jin et al., 2025; Qin et al., 2025); (iii) **control loops and goal management**, which execute objectives, monitor progress, reflect on outcomes, and recover from failure (Yao et al., 2023a; Shinn et al., 2023; Madaan et al., 2023; Sun et al., 2023); (iv) **tool allocation**, which selects and sequences callable resources from large API spaces (Qin et al., 2023; Yang et al., 2026; Qian et al., 2025; Wang et al., 2025a); and (v) **skill orchestration**, which discovers, retrieves, composes, and delegates reusable procedures from growing skill libraries (Xu and Yan, 2026; Zheng et al., 2025; Li et al., 2026a; Zhou et al., 2026). Modularization has consequently become the field’s common design language: representations vary, but explicit interfaces make harness components independently substitutable and optimizable (Chen et al., 2026b; Ning et al., 2026; Pan et al., 2026a; Lin et al., 2026; Lee et al., 2026c). For example, HarnessX factorizes a runtime into prompts, tools, memory, and control flow, whereas Code-as-Agent-Harness organizes it into interface, mechanism, and multi-agent-scaling layers (Chen et al., 2026b; Ning et al., 2026). We therefore adopt the compact factorization $\mathbf{h} = (\mathbf{M}, \mathbf{P}, \mathbf{A}, \mathbf{F})$, where the four modules construct the memory view, form the next directive, advance the action loop, and orchestrate tools and skills, respectively; this provides a minimal executable contract while leaving each module internally extensible (Hu et al., 2026b; Cao et al., 2025; Yao et al., 2023a; Xu and Yan, 2026).

Harness Optimization. Once modularized, harness construction becomes an optimization problem over executable artifacts rather than a fixed engineering choice (Khattab et al., 2023; Yuksekgonul et al., 2024; Hu et al., 2024b; Shang et al., 2024). The optimization scope has expanded from prompts and declarative LM pipelines to workflow graphs, role assignments, and complete harness code,

Table 1: Harness optimization paradigms. *Construction* distinguishes harnesses found by ahead-of-time search, ahead-of-time harnesses subsequently edited with test-time feedback, and harnesses generated just in time for each task. *Instance synthesis*, *Harness model*, *Learned repair*, and *Online evolution* indicate whether a method directly synthesizes an instance-specific harness, trains the generator, learns repair from failed execution trajectories, and continues improving after deployment, respectively.

Method	Construction	Instance synthesis	Harness model	Learned repair	Online evolution
AutoHarness (Lou et al., 2026)	AOT (search)	✗	✗	✗	✗
Meta-Harness (Lee et al., 2026c)	AOT (search)	✗	✗	✗	✗
AHE (Lin et al., 2026)	AOT (search)	✗	✗	✗	✗
Adaptive AH (Liu et al., 2026c)	AOT (test-time editing)	✗	✗	✗	✓
TTHE (Nie et al., 2026b)	AOT (test-time editing)	✗	✗	✗	✓
RHI (Lee et al., 2026a)	AOT (test-time editing)	✗	✗	✗	✓
Harness-R1 (Shao et al., 2026)	AOT (test-time editing)	✗	✓	✓	✓
JIT-Agent (ours)	JIT	✓	✓	✓	✓

allowing feedback to revise both local instructions and global control structure (Khattab et al., 2023; Zhang et al., 2024; Hu et al., 2024b; Shang et al., 2024; Lee et al., 2026c). **Closed-loop evolution.** At the harness level, recent methods largely instantiate a diagnose–propose–validate loop: execution traces expose behavioral weaknesses, candidate edits alter prompts, memory, tools, middleware, or control flow, and validation or preference signals determine which changes persist (Lee et al., 2026c; Lin et al., 2026; Zhang et al., 2026b; Pan et al., 2026b; Chen et al., 2026b). Recent work further extends this loop toward recursive, hierarchical, and collaborative self-improvement, while constraining component-wise evolution to improve transfer (Lee et al., 2026a; Zhou, 2026; Nie et al., 2026a; Zhang et al., 2026c). Their supervision ranges from textual gradients and trace-grounded observability to regression tests, retrospective self-preference, and typed primitive substitution, but the common product is a persistently improved executable scaffold (Yuksekgonul et al., 2024; Lin et al., 2026; Zhang et al., 2026b; Pan et al., 2026b; Chen et al., 2026b). **AOT versus JIT.** Although such harnesses may transfer, these methods primarily optimize a durable artifact against an accumulated experience distribution and are therefore ahead-of-time relative to the exact structure of a new instance (Zhang et al., 2024, 2025a; Yuan et al., 2024; Chen et al., 2023). We instead amortize harness search into JIT-Agent: the generator synthesizes a task-conditioned harness at inference time and can further revise it from execution feedback, shifting optimization from repeated artifact search toward learned just-in-time construction (Meng et al., 2026b; Zhou et al., 2026; Ning et al., 2026; Pan et al., 2026a; Xu and Yan, 2026). Table 1 makes this distinction explicit.

3 Unified Harness Codebase

3.1 Modularized Harness Design Space

An agent harness is the operational layer that turns a foundation model into a closed-loop agent (Meng et al., 2026a; Zhou et al., 2026): it determines what from prior interaction is retained (Hu et al., 2026b), how intermediate directives are formed (Cao et al., 2025), which external capabilities are exposed at each stage (Xu and Yan, 2026; Yang et al., 2026), and how control advances. Harness generation is therefore defined over programs rather than unconstrained text. We distinguish the raw generation space $\mathcal{G}$, syntactically valid harnesses $\mathcal{H}^{\text{syn}}$, protocol-compliant harnesses $\mathcal{H}_{\Pi}$, and their executable subset $\mathcal{H}_{\Pi}^{\text{exec}}$, with $\mathcal{G} \supseteq \mathcal{H}^{\text{syn}} \supseteq \mathcal{H}_{\Pi} \supseteq \mathcal{H}_{\Pi}^{\text{exec}}$. The fixed protocol Π specifies module schemas and interfaces, their lifecycle, validation rules, and the shared execution semantics. It removes incidental variation in language and runtime while retaining the principal operational choices that distinguish existing harnesses (Wu et al., 2023; Microsoft, 2024; LangGraph, 2025; Luo et al., 2025).

Harness Composability

Composability has become a first-class concern in modern harness design. DeepSeek Harness, for example, adopts an “Everything is a Plugin” architecture, representing the runtime as modules connected through explicit dependency relationships (Shi et al., 2026). Composability is equally central to JIT generation: it turns harness construction from unconstrained program synthesis into assembly over a typed, recombinable design space.

JIT-Agent makes these choices explicit through four interoperable modules: *how history is compressed*, *how local intent is formed*, *how tools and skills are orchestrated*, and *how control advances*. Let τ be a task, π_ψ a frozen backbone executor, and C_τ the capability registry available (e.g., tools, APIs, skills) to that task. Running a harness $\mathbf{h}$ with π_ψ induces a closed-loop trajectory

$$\xi \sim \text{Rollout}(\tau, \pi_\psi, \mathbf{h}, C_\tau; \Pi) = (\mathbf{s}_1, e_1, \mathbf{o}_1, \dots, \mathbf{s}_T, e_T, \mathbf{o}_T), \quad (1)$$

where $\mathbf{s}_t \in \mathcal{S}$ is the maintained controller state, e_t is the emitted tool call or terminal output, $\mathbf{o}_t \in \mathcal{O}$ is the resulting observation, and T is the protocol- or budget-bounded stopping time. Our key assumption is that every $\mathbf{h} \in \mathcal{H}_\Pi$ admits the modular factorization

$$\mathbf{h} = (\mathbf{M}, \mathbf{P}, \mathbf{A}, \mathbf{F}) \in \mathfrak{M} \times \mathfrak{P} \times \mathfrak{A} \times \mathfrak{F}, \quad (2)$$

where $\mathbf{M}$, $\mathbf{P}$, $\mathbf{A}$, and $\mathbf{F}$ denote memory, planning, action, and capability-orchestration modules, and $\mathfrak{M}$, $\mathfrak{P}$, $\mathfrak{A}$, and $\mathfrak{F}$ are their protocol-compatible implementation spaces. The tuple follows this conceptual decomposition; at runtime, its dependency order is $\mathbf{M} \rightarrow \mathbf{P} \rightarrow \mathbf{F} \rightarrow \mathbf{A}$. All modules operate against the same frozen π_ψ , whose common runtime dependence is suppressed below.

Position

Production harnesses such as Codex, Claude Code and DeepSeek Harness expose substantially richer mechanisms than our four-module instantiation (Shi et al., 2026; Anthropic, 2025; OpenAI, 2025). That gap is deliberate. This work is not to ask a model to reproduce an entire production runtime, but to establish a more fundamental result: even compact harnesses, generated just in time, can yield substantial gains. Besides, we believe the JIT paradigm is complementary to production harness engineering. As models increasingly participate in the design and revision of their own runtimes, the training recipe for harness intelligence will matter at larger scales as well. We view the present design space as a starting point.

The protocol maintains both the immutable event history $\xi_{<t}$ and the mutable controller state $\mathbf{s}_t$, which interact as

$$\mathbf{v}_t = \mathbf{M}(\xi_{<t}, \mathbf{s}_t) \in \mathcal{V}, \quad \text{history} \rightarrow \text{view}, \quad (3)$$

$$\mathbf{d}_t = \mathbf{P}(\tau, \mathbf{s}_t, \mathbf{v}_t) \in \mathcal{D}_{\text{dir}}, \quad \text{view} \rightarrow \text{local directive}, \quad (4)$$

$$C_t = \mathbf{F}(C_\tau, \mathbf{s}_t, \mathbf{v}_t, \mathbf{d}_t) \subseteq C_\tau, \quad \text{directive-conditioned capability orchestration}, \quad (5)$$

$$(\mathbf{s}_{t+1}, e_t) = \mathbf{A}(\mathbf{s}_t, \tau, \mathbf{v}_t, \mathbf{d}_t, C_t) \in \mathcal{S} \times \mathcal{A}, \quad \text{control update and action emission}, \quad (6)$$

where $\mathcal{V}$ and $\mathcal{D}_{\text{dir}}$ are the view and directive spaces, and $\mathcal{A} = \mathcal{U} \sqcup \mathcal{Y}$ is the disjoint union of executable calls and terminal outputs. The registry C_τ contains callable tools (e.g., bash tools, APIs, and MCPs (Sarkar and Sarkar, 2025)) as well as higher-level agent skills (Xu and Yan, 2026; ClaudeCode, 2025). A harness without an explicit planner remains type-consistent through the null directive $\mathbf{d}_0 \in \mathcal{D}_{\text{dir}}$, returned by $\mathbf{P}_0$ at every step. That is, memory constructs a view of realized history, planning converts that view into a local directive, capability orchestration activates the relevant external tools or agent skills, and the action module consumes the assembled context to both update the controller state and emit the next action. The kernel then interprets the emitted action by

$$\mathbf{o}_t = \begin{cases} \text{Exec}(e_t; C_t), & e_t \in \mathcal{U}, \\ \perp, & e_t \in \mathcal{Y}, \end{cases} \quad \xi_{\leq t} = \xi_{<t} \oplus (\mathbf{s}_t, e_t, \mathbf{o}_t), \quad (7)$$

Table 2: Seed bank $\mathcal{B}_0$: 13 hand-written harnesses instantiating the four-module protocol Π . Each row is a complete harness; columns follow the conceptual order of memory, planning, action, and capability orchestration.

Harness	♣ Memory $M \in \mathfrak{M}$	♦ Planning $P \in \mathfrak{P}$	♥ Action $A \in \mathfrak{A}$	♠ Capability Orchestration $F \in \mathfrak{F}$
ReAct [83]	FullHistory	No explicit planner	ReAct	Full registry
PLAN-AND-EXECUTE [16]	FullHistory	Linear roadmap	ReAct	Full registry
ReSUM [77]	ReSum memory	No explicit planner	ReAct	Full registry
FLASH-SEARCHER [60]	FullHistory	DAG planning	ReAct	Full registry
GAM [79]	GAM retrieval	DAG	ReAct	Full registry
MEMOBRAIN [59]	Reasoning graph	No explicit planner	Marker-guided execution	Full registry
AGGAGENT [32]	Isolated rollout histories	No explicit planner	Multi-rollout aggregation	Full registry
OAGENT [100]	Coordinator history	No explicit planner	Ensemble voting	Full registry
AGENTFOLD [84]	AgentFold memory	DAG	ReActFold	Full registry
HiAGENT [22]	Hierarchical memory	No explicit planner	ReAct	Full registry
DEEPAGENT [34]	Three-tier memory	No explicit planner	Marker-guided execution	Tool search
ROMA [1]	Context isolation	Atomizer + DAG	Recursive execution	Full registry
AORCHESTRA [63]	Context isolation	Atomizer + DAG	Recursive execution	Agent Delegation

where $\text{Exec} : \mathcal{U} \times 2^{C_\tau} \rightarrow \mathcal{O}$ is the shared execution kernel, $\perp \in \mathcal{O}$ is the terminal null observation, and $\oplus$ appends an event to the trajectory. Execution starts from $\xi_{<1} = \emptyset$ and a protocol-defined $s_1 \in \mathcal{S}$, and terminates when $e_t \in \mathcal{Y}$, yielding $y = e_t \in \mathcal{Y}$. This factorization turns otherwise heterogeneous programs into comparable coordinates in $\mathfrak{M} \times \mathfrak{P} \times \mathfrak{A} \times \mathfrak{F}$:

- **Canonical ReAct** can be written as $\mathbf{h}_{\text{ReAct}} = (\mathbf{M}_{\text{full}}, \mathbf{P}_\emptyset, \mathbf{A}_{\text{react}}, \mathbf{F}_{\text{all}})$, where $\mathbf{A}_{\text{react}}$ is the standard ReAct loop, $\mathbf{P}_\emptyset$ denotes the absence of an explicit planner, $\mathbf{M}_{\text{full}}$ keeps the running history without context management, and $\mathbf{F}_{\text{all}}$ exposes the full tool/skill registry.
- **Engineered ReAct variants** like Codex (OpenAI, 2025) and OpenCode (OpenCode-AI, 2025) still fit the same scaffold: $(\mathbf{M}_{\text{compact}}, \mathbf{P}_{\text{todo}}, \mathbf{A}_{\text{react}}, \mathbf{F}_{\text{all}})$, where the action kernel remains ReAct-style, but $\mathbf{M}_{\text{compact}}$ compresses history near the context limit and $\mathbf{P}_{\text{todo}}$ maintains an explicit task todo list.
- **Recursive agents.** Recursive architectures such as ROMA (Alzu'bi et al., 2026), AOchestra (Ruan et al., 2026), and Recursive Language Models (RLM) (Zhang et al., 2026a) are likewise captured by choices such as $\mathbf{h}_{\text{rec}} = (\mathbf{M}_{\text{subproblem}}, \mathbf{P}_{\text{decomp}}, \mathbf{A}_{\text{rec}}, \mathbf{F}_{\text{route}})$. In such systems, the main orchestrator may spawn subagents through $\mathbf{A}_{\text{rec}}$, maintain a todo-style decomposition in $\mathbf{P}_{\text{decomp}}$, keep agent contexts isolated in $\mathbf{M}_{\text{subproblem}}$, and allocate tools or skills to each subagent through $\mathbf{F}_{\text{route}}$.

Given this modularized design space, we next introduce HARNESSFACTORY, whose role is to test the expressiveness of $\mathcal{H}_\Pi$ and furnish diverse source material for JIT-Agent’s meta-harness design.

3.2 HarnessFactory

Under the shared protocol Π and a common kernel, HARNESSFACTORY re-implements 13 representative contemporary agentic scaffolds: REACT (Yao et al., 2023b), PLAN-AND-EXECUTE (Erdogan et al., 2025b), RESUM (Wu et al., 2026), FLASH-SEARCHER (Qin et al., 2025), GENERAL AGENTIC MEMORY (GAM) (Yan et al., 2025), MEMOBRAIN (Qian et al., 2026), AGGAGENT (Lee et al., 2026d), OAGENT (Zhu et al., 2025), AGENTFOLD (Ye et al., 2025), HiAGENT (Hu et al., 2024a), DEEPAGENT (Li et al., 2026b), ROMA (Alzu'bi et al., 2026), and AORCHESTRA (Ruan et al., 2026). They provide heterogeneous memory, planning, action, and capability-orchestration strategies. The seed bank $\mathcal{B}_0$ therefore contains $K_0 = 13$ protocol-compatible harnesses. Table 2 gives the full inventory.

The bank serves a dual role: harnesses sampled from $\mathcal{B}_0$ anchor Stage-I synthesis, while later archive states supply the prior population against which new designs are evaluated. As the system operates, $\mathcal{B}_0$ grows into $\mathcal{B}_n \supseteq \mathcal{B}_0$, whose entries associate each retained harness with its task and observed reward, latency, and cost; this evolution is formalized in Section 4.3.

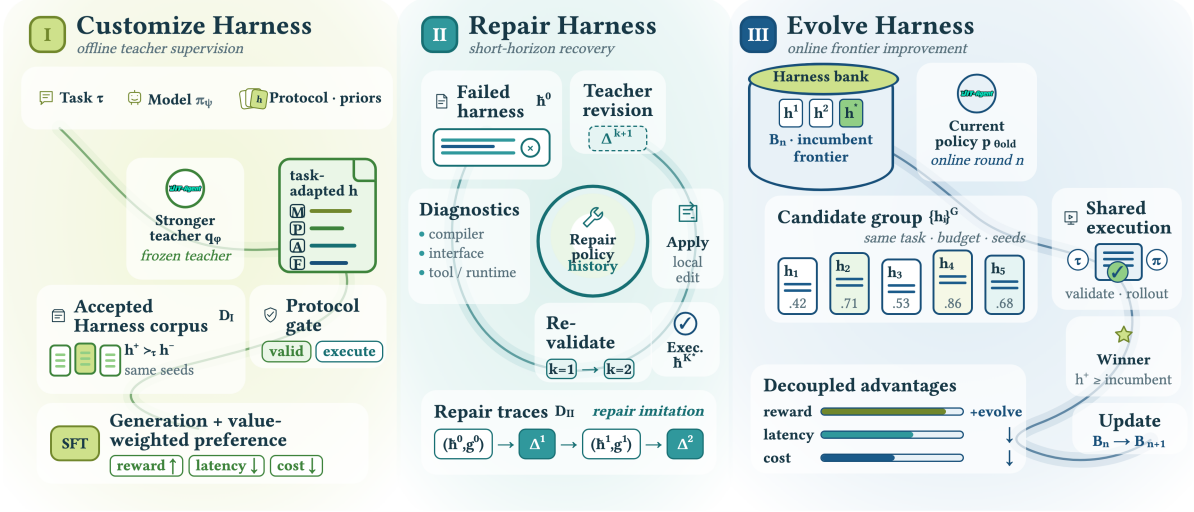


Figure 3: Training pipeline of JIT-Agent. Stage I learns task-conditioned harness customization. Stage II converts failed harnesses and execution diagnostics into bounded repair trajectories. Stage III performs online evolution by comparing candidate harnesses against the incumbent bank, optimizing decoupled reward, latency, and cost advantages, and retaining frontier-improving designs.

4 Training Pipeline

Training follows the inference-time lifecycle of JIT-Agent: first synthesize a task-conditioned harness, then recover unstable generations, and finally learn to improve from stronger archive states online. Let $\tau \sim \mathcal{D}_{\text{task}}$ denote a task drawn from the training distribution, C_τ its capability registry, and $\mathcal{E}_\tau$ a small reference context from the harness bank. We define the generation context as $\mathbf{c}_\tau = (\tau, \Pi, C_\tau, \mathcal{E}_\tau)$ and sample harnesses from $p_\theta(\mathbf{h} \mid \mathbf{c}_\tau)$. A frozen executor $\pi_\psi \sim \mathcal{M}$ then runs each harness for validation and utility measurement. The common objective is

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\tau \sim \mathcal{D}_{\text{task}}, \pi_\psi \sim \mathcal{M}, \mathbf{h} \sim p_\theta(\cdot \mid \mathbf{c}_\tau)} [U(\tau, \pi_\psi, \mathbf{h})], \quad (8)$$

where U evaluates the trajectory induced by the model–harness pair using task reward, latency, and monetary cost. Because p_θ generates in $\mathcal{G}$ rather than satisfying executability by construction, every output is checked by the protocol validator. We use $\text{Valid}_\Pi(\mathbf{h}; \tau, \pi_\psi, C_\tau) \in \{0, 1\}$ for its validity flag; failed checks additionally return a structured diagnostic report. We instantiate Equation (8) with offline supervision in Stage I, repair-trajectory learning in Stage II, and online policy improvement in Stage III.

4.1 Stage I: Customizing Harness

Data Preparation. Stage I uses a frozen, stronger teacher q_ϕ to synthesize task-adapted harnesses under the fixed four-module protocol. For each task, three reference scaffolds are sampled from the task-type-matched subset $\mathcal{B}_0^{(d(\tau))}$ of the seed bank,

$$\mathcal{E}_\tau = \{\mathbf{h}^{(1)}, \mathbf{h}^{(2)}, \mathbf{h}^{(3)}\} \sim \text{Sample}_3(\mathcal{B}_0^{(d(\tau))}), \quad \mathbf{h}^{\text{teach}} \sim q_\phi(\cdot \mid \mathbf{c}_\tau), \quad (9)$$

where $d(\tau)$ denotes the task type. The teacher receives the task, protocol, capability registry, and sampled scaffolds. A generation is retained only if it passes protocol validation and execution checks, yielding

$$\mathcal{D}_I = \left\{ (\tau, \pi_\psi, C_\tau, \mathcal{E}_\tau, \mathbf{h}^{\text{teach}}) \mid \text{Valid}_\Pi(\mathbf{h}^{\text{teach}}; \tau, \pi_\psi, C_\tau) = 1 \right\}. \quad (10)$$

Stage I draws its training tasks from different sources, including (Wu et al., 2025; Shi et al., 2025; Zhang et al., 2026d; Bai et al., 2026; Burgess et al., 2026; Song et al., 2026), as well as a portion of synthesized tasks.

Training Setup. Stage I uses two coupled objectives. The first is a standard supervised fine-tuning objective over accepted teacher generations:

$$\mathcal{L}_I^{\text{gen}}(\theta) = -\mathbb{E}_{(\tau, \pi_\psi, C_\tau, \mathcal{E}_\tau, \mathbf{h}^{\text{teach}}) \sim \mathcal{D}_I} \sum_{j=1}^{|\mathbf{h}^{\text{teach}}|} \log p_\theta(y_j^{\text{teach}} | y_{<j}^{\text{teach}}, \mathbf{c}_\tau), \quad (11)$$

where y_j^{teach} is the j -th target token. This teaches JIT-Agent to map a task and a small reference context directly to a protocol-compliant harness. Protocol compliance alone is insufficient: executable harnesses can still differ substantially in task reward, latency, and monetary cost. We therefore compare candidates under the same backbone and evaluation seeds, retaining a preference only when reward improves without degrading either efficiency axis and at least one efficiency gain is strict.

$$\begin{aligned} \mathbf{h}^+ \succ_\tau \mathbf{h}^- &\iff r^+ > r^- \wedge \ell^+ \leq \ell^- \wedge \kappa^+ \leq \kappa^- \wedge (\ell^+ < \ell^- \vee \kappa^+ < \kappa^-), \\ \Delta_{\text{val}}(\tau; \mathbf{h}^+, \mathbf{h}^-) &= \alpha_r(r^+ - r^-) + \alpha_\ell[\ell^- - \ell^+]_+ + \alpha_\kappa[\kappa^- - \kappa^+]_+, \end{aligned} \quad (12)$$

where $(r^\pm, \ell^\pm, \kappa^\pm)$ are repeated-rollout averages for $(\mathbf{h}^+, \mathbf{h}^-)$, $[x]_+ = \max(x, 0)$, and $\alpha_r, \alpha_\ell, \alpha_\kappa \geq 0$ control the three value gaps. Let $\mathcal{D}_I^{\text{pref}}$ collect the resulting preference tuples. We optimize the reference-anchored objective

$$\mathcal{L}_I^{\text{pref}}(\theta) = -\mathbb{E}_{(\tau, \pi_\psi, C_\tau, \mathcal{E}_\tau, \mathbf{h}^+, \mathbf{h}^-) \sim \mathcal{D}_I^{\text{pref}}} \left[\Delta_{\text{val}} \cdot \log \sigma \left(\beta_{\text{pref}} \log \frac{p_\theta(\mathbf{h}^+ | \mathbf{c}_\tau)}{p_\theta(\mathbf{h}^- | \mathbf{c}_\tau)} - \beta_{\text{pref}} \log \frac{p_{\text{ref}}(\mathbf{h}^+ | \mathbf{c}_\tau)}{p_{\text{ref}}(\mathbf{h}^- | \mathbf{c}_\tau)} \right) \right]. \quad (13)$$

Here σ is the logistic sigmoid, $\beta_{\text{pref}} > 0$ controls preference sharpness, and p_{ref} is the frozen Stage-I SFT checkpoint; in the log ratios, $\log p(\mathbf{h} | \cdot)$ denotes length-normalized sequence log-likelihood. The full Stage-I objective is $\mathcal{L}_I(\theta) = \mathcal{L}_I^{\text{gen}}(\theta) + \lambda_{\text{pref}} \mathcal{L}_I^{\text{pref}}(\theta)$ with $\lambda_{\text{pref}} \geq 0$. Generation imitation establishes protocol-valid structure, while preference learning biases the model toward harnesses that are simultaneously more effective and more efficient.

4.2 Stage II: Repairing Harness

Data Preparation. Stage I optimizes for protocol compliance, but executability is still not guaranteed: some generated harnesses fail static or runtime validation and therefore never enter $\mathcal{D}_I$. Rather than discarding these failures, Stage II converts them into repair supervision. Let

$$\mathcal{D}_I^{\text{fail}} = \left\{ (\tau, \pi_\psi, C_\tau, \mathcal{E}_\tau, \tilde{\mathbf{h}}^{(0)}, \mathbf{g}^{(0)}) \mid \tilde{\mathbf{h}}^{(0)} \notin \mathcal{H}_\Pi^{\text{exec}} \right\}, \quad (14)$$

where $\tilde{\mathbf{h}}^{(0)}$ is a failed Stage-I harness and $\mathbf{g}^{(0)}$ is its diagnostic report, including compiler errors, interface mismatches, tool-call failures, and runtime exceptions. For each failure, the teacher proposes a structured revision $\Delta^{(k+1)} \in \mathcal{P}$ from the patch space $\mathcal{P}$, and Apply deterministically applies it to the current harness:

$$\tilde{\mathbf{h}}^{(k)}, \mathbf{g}^{(k)} \longrightarrow \Delta^{(k+1)} \longrightarrow \tilde{\mathbf{h}}^{(k+1)} = \text{Apply}(\tilde{\mathbf{h}}^{(k)}, \Delta^{(k+1)}), \quad k \geq 0. \quad (15)$$

After each revision, validation produces the next report $\mathbf{g}^{(k+1)}$. We retain only trajectories that become executable within two repair rounds. Writing

$$K^* = \min \left\{ k \in \{1, 2\} : \text{Valid}_\Pi(\tilde{\mathbf{h}}^{(k)}; \tau, \pi_\psi, C_\tau) = 1 \right\}, \quad (16)$$

Stage-II supervision is constructed only when K^* is well defined. This keeps the corpus focused on realistic, locally recoverable failures instead of those requiring wholesale redesign.

Training Setup. Stage II then trains JIT-Agent to imitate successful repair transitions rather than one-shot synthesis. For each retained trajectory, let

$$\mathcal{D}_{\text{II}} = \left\{ (\tau, \pi_\psi, C_\tau, \mathcal{E}_\tau, \mathcal{R}_{K^*}) \mid (\tau, \pi_\psi, C_\tau, \mathcal{E}_\tau, \tilde{\mathbf{h}}^{(0)}, \mathbf{g}^{(0)}) \in \mathcal{D}_1^{\text{fail}}, K^* \text{ exists} \right\}, \quad (17)$$

where $\mathcal{R}_{K^*} = \{(\tilde{\mathbf{h}}^{(j)}, \mathbf{g}^{(j)}, \Delta^{*(j+1)})\}_{j=0}^{K^*-1}$ and each $\Delta^{*(j+1)}$ is the teacher revision along a repair trajectory that eventually becomes executable. The repair objective conditions on the entire history,

$$\mathcal{L}_{\text{II}}(\theta) = -\mathbb{E}_{\mathcal{D}_{\text{II}}} \sum_{k=0}^{K^*-1} \log p_\theta \left(\Delta^{*(k+1)} \mid \mathbf{c}_\tau, \{(\tilde{\mathbf{h}}^{(j)}, \mathbf{g}^{(j)})\}_{j=0}^k \right). \quad (18)$$

Because $K^* \leq 2$, the resulting supervision targets exactly the short-horizon repair regime that matters at deployment time: given a nearly-correct but unstable harness, JIT-Agent learns to use execution feedback to produce a small number of high-leverage revisions that restore protocol-valid execution.

4.3 Stage III: Learning to Evolve Harness

Stage III treats *test-time harness evolution* itself as a trainable capability. The goal is not merely to recover or imitate previously observed harness designs, but to optimize JIT-Agent so that, at test time, it can repeatedly propose harnesses that surpass prior designs, become stronger future references, and keep pushing the harness frontier forward. We term the resulting online objective **Evolutionary Group-Decoupled Policy Optimization (Evo-GDPO)**, inspired by (Liu et al., 2026b).

Data Preparation. Stage III training examples are constructed from three synchronized sources: a task instance, a small set of prior high-quality harness designs, and fresh execution feedback collected online under that same task. Concretely, at online round n , we sample a task τ , retrieve a small reference set $\mathcal{E}_{\tau,n}$ from the current harness bank $\mathcal{B}_n$, and form $\mathbf{c}_{\tau,n} = (\tau, \Pi, C_\tau, \mathcal{E}_{\tau,n})$. The model proposes candidates that are executed beside these prior designs under the same frozen executor $\pi_\psi \sim \mathcal{M}$, budget, and evaluation seeds. Within $\mathcal{E}_{\tau,n}$, the highest-reward harness, with ties broken by lower latency and then lower cost, supplies a single incumbent whose statistics are denoted by (b_r, b_ℓ, b_κ) .

Training Setup. Given this context, Evo-GDPO first samples a group of candidate harnesses from the current policy:

$$\mathbf{h}_i \stackrel{\text{i.i.d.}}{\sim} p_{\theta_{\text{old}}}(\cdot \mid \mathbf{c}_{\tau,n}), \quad i = 1, \dots, G, \quad (19)$$

where $G > 1$ is the group size and θ_{old} is the rollout-policy snapshot. Every candidate is validated before execution; a candidate that remains invalid after bounded repair receives the minimum task reward, while its repair latency and cost remain part of the measured efficiency. For executable candidates, r_i , $\bar{\ell}_i$, and $\bar{\kappa}_i$ denote reward, mean latency, and mean monetary cost over repeated rollouts. The reward channel is primary, while the efficiency channels activate only when the candidate preserves incumbent reward.

$$\begin{aligned} R_i^{\text{rew}} &= r_i + \lambda_{\text{evo}} [r_i - b_r]_+, \\ R_i^{\text{lat}} &= \mathbb{I}[r_i \geq b_r] [b_\ell - \bar{\ell}_i]_+, \\ R_i^{\text{cost}} &= \mathbb{I}[r_i \geq b_r] [b_\kappa - \bar{\kappa}_i]_+. \end{aligned} \quad (20)$$

Here $\lambda_{\text{evo}} \geq 0$ controls the bonus and $\mathbb{I}[\cdot]$ is the indicator function. Evo-GDPO normalizes the three signals separately before merging them, preventing their numerical scales from overwhelming one

another. A second batch-level normalization stabilizes optimization across tasks.

$$\begin{aligned}
A_i^m &= \frac{R_i^m - \text{mean}(\{R_j^m\}_{j=1}^G)}{\text{std}(\{R_j^m\}_{j=1}^G) + \epsilon_{\text{num}}}, \quad m \in \{\text{rew}, \text{lat}, \text{cost}\}, \\
A_i^\Sigma &= w_{\text{rew}} A_i^{\text{rew}} + w_{\text{lat}} A_i^{\text{lat}} + w_{\text{cost}} A_i^{\text{cost}}, \quad w_{\text{rew}} > w_{\text{lat}} + w_{\text{cost}}, \\
\widehat{A}_i^\Sigma &= \frac{A_i^\Sigma - \text{mean}_{\text{batch}}(A^\Sigma)}{\text{std}_{\text{batch}}(A^\Sigma) + \epsilon_{\text{num}}}.
\end{aligned} \tag{21}$$

The nonnegative weights sum to one, and the stated inequality makes task reward dominant; $\epsilon_{\text{num}} > 0$ is a numerical stabilizer in both denominators above. The final policy update is a PPO-style clipped objective driven by this aggregated advantage. Relative to standard GRPO-style training, the model is rewarded not simply for being good within the sampled group, but for overtaking prior harness designs and doing so more efficiently whenever reward quality is preserved.

$$\begin{aligned}
\mathcal{L}_{\text{III}}^{\text{Evo-GDPO}}(\theta) &= -\mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|\mathbf{h}_i|} \sum_{j=1}^{|\mathbf{h}_i|} \min \left(\rho_{i,j}(\theta) \widehat{A}_i^\Sigma, \text{clip}(\rho_{i,j}(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) \widehat{A}_i^\Sigma \right) \right] \\
&\quad + \beta_{\text{KL}} \mathbb{E} [\text{KL}(p_\theta \| p_{\text{ref}})], \\
\rho_{i,j}(\theta) &= \frac{p_\theta(y_{i,j} | y_{i,<j}, \mathbf{c}_{\tau,n})}{p_{\theta_{\text{old}}}(y_{i,j} | y_{i,<j}, \mathbf{c}_{\tau,n})}.
\end{aligned} \tag{22}$$

The expectation is over sampled tasks, frozen executors, and candidate groups from $p_{\theta_{\text{old}}}$; $|\mathbf{h}_i|$ is the harness token length, $\epsilon_{\text{clip}} > 0$ is the clipping radius, $\beta_{\text{KL}} \geq 0$, and p_{ref} is the frozen Stage-II checkpoint used for the token-level KL penalty. After optimization, we update the harness bank conservatively: a candidate is retained only if it matches or exceeds the current reward frontier and then strictly improves at least one frontier dimension—either reward itself, latency, or cost. During training, this feedback updates both θ and $\mathcal{B}_n$; at deployment, θ remains frozen.

5 Inference Architecture

JIT-Agent supports two inference modes: *static inference* and *streaming inference*; they differ in whether experience is discarded after the task or retained to support subsequent tasks.

Static inference. In this mode, we introduce a lightweight form of test-time scaling: JIT-Agent generates N harnesses in parallel, selects one of them, and executes only the selected harness. This increases candidate diversity without increasing the number of environment rollouts. The selected harness follows the same validation and bounded-repair procedure described above.

Streaming inference. Streaming inference mode is designed to carry useful experience forward across a sequence of tasks. For the n -th task τ_n , JIT-Agent retrieves from the current bank $\mathcal{B}_n$, generates and selects a harness $\mathbf{h}_n^\dagger$, and executes it once. The resulting environment feedback is then used only to determine whether this experience should update the bank:

$$\begin{aligned}
\xi_n &\sim \text{Rollout}(\tau_n, \pi_\psi, \mathbf{h}_n^\dagger, C_{\tau_n}; \Pi), \\
\mathbf{m}_n &= \text{Eval}(\xi_n), \\
\mathcal{B}_{n+1} &= \text{Update}_{\text{III}}(\mathcal{B}_n; \tau_n, \mathbf{h}_n^\dagger, \mathbf{m}_n), \\
\mathcal{E}_{\tau_{n+1}, n+1} &= \text{Retrieve}(\tau_{n+1}; \mathcal{B}_{n+1}),
\end{aligned} \tag{23}$$

where $\mathbf{m}_n = (r_n, \bar{\ell}_n, \bar{\kappa}_n)$ contains reward, mean latency, and mean monetary cost. Following the Stage-III retention rule, $\text{Update}_{\text{III}}$ leaves $\mathcal{B}_n$ unchanged when the completed harness provides no admissible improvement. Otherwise, the retained harness becomes a potential reference for later tasks. Streaming inference therefore transfers prior experience through the evolving harness bank, without injecting environment feedback into the current rollout or updating model parameters.

6 Experiments and Analysis

6.1 Experiment Setup

Evaluation Benchmarks. We evaluate JIT-Agent on nine benchmarks grouped into four task types:

- **Deep Research:** BrowseComp-Plus (Chen et al., 2025b) (accuracy), DeepSearchQA (Gupta et al., 2026) (answer F1), and xBench-DeepSearch (xBench-DS) (Chen et al., 2025a) (accuracy).
- **Daily Work:** AgentIF-Oneday (Chen et al., 2026a) (normalized weighted rubric score) and Pinch-Bench (PinchBench, 2026) (average score).
- **Planning:** DeepPlanning-Shopping (cart match rate) and DeepPlanning-Travel (composite constraint score) subsets from (Zhang et al., 2026d).
- **Workspace:** OfficeBench (Wang et al., 2024) and OdysseyBench (Wang et al., 2025b) (task success rate for both).

Together, these benchmarks cover evidence-intensive search, long-horizon instruction following, constraint-aware planning, and multi-application workspace execution. We report all metrics on a 0–100 scale; higher is better.

Backbone Models. JIT-Agent is trained based on Qwen3.6-27B. We primarily instantiate JIT-Agent with GLM-5.2 (Z.ai, 2026) and DeepSeek-V4-Flash-Preview (DeepSeek-AI, 2026). These backbones provide complementary operating points: GLM-5.2 is a strong open long-horizon model, whereas DeepSeek-V4-Flash emphasizes inference efficiency. To test the generalization, additional models from Qwen3.6, MIMO-V2.5 are also used.

Baselines. We use two complementary baseline groups. First, the broad comparison includes vanilla Qwen3.7-Plus (Qwen Team, 2026), GLM-5.2, DeepSeek-V4-Flash/Pro (both preview version), Kimi K2.7 Code (Moonshot AI, 2026), GPT-5.6 (OpenAI, 2026), and Gemini 3.1 Pro/3.5 Flash (Google DeepMind, 2026a,b). Second, holding the backbone fixed, we compare JIT-Agent with five reusable agent harnesses: CLAUDE CODE (Anthropics, 2025), CODEX (OpenAI, 2025), OPENCODE (OpenCode-AI, 2025), HERMES (Nous Research, 2026), and NANO BOT (HKUDS, 2026). The first group measures end-to-end competitiveness, while the second isolates the contribution of harness design from that of model weights.

6.2 Main Results

Table 3 compares JIT-generated harnesses with vanilla agentic backbones and frontier model baselines. We report three deep research benchmarks, two daily work benchmarks, two planning benchmarks, and two workspace benchmarks.

Consistent gains over the same backbone. Across all 18 directly matched backbone–benchmark pairs, replacing the default scaffold with a JIT-generated harness improves performance. On GLM-5.2, the nine-benchmark average rises from 74.1 to 81.8 (+7.7 points); on DeepSeek-V4-Flash, it increases from 66.7 to 75.5 (+8.8 points). The largest gains occur on tasks that require sustained state management and constraint tracking: DeepSeek-V4-Flash improves by 24.8 points on DeepPlanning-Shopping (59.1 → 83.9), while GLM-5.2 gains 20.2 points on DeepPlanning-Travel (62.8 → 83.0). Search and open-ended interaction also benefit substantially, including +11.9 on xBench-DS and +8.9 on DeepSearchQA for DeepSeek-V4-Flash. These broad gains indicate that harness adaptation changes more than prompt style: it improves how the backbone allocates context, decomposes long-horizon objectives, and coordinates external actions.

Competitiveness with frontier models. The JIT-equipped systems attain the best result in eight of the nine benchmark columns despite using open backbones. In particular, JIT-Agent + GLM-5.2 ranks first on seven benchmarks, reaching 93.9 on DeepSearchQA, 69.9 on AgentIF, and 93.3 on

Table 3: Main results across nine agentic benchmarks. Rows prefixed by JIT-Agent use the same backbone as their vanilla counterpart but replace its default scaffold with a JIT-generated harness. Scores are reported on a 0–100 scale; higher is better. Top-1, Top-2, and Top-3 results in each column are highlighted in soft green, yellow-green, and yellow, respectively; Top-1 is also bolded. Missing evaluations are denoted by “—”.

Model	Deep Research			Daily Work		Planning		Workspace	
	BC+	DSQA	xBench	AgentIF	PinchBench	Shop	Travel	Office	Odyssey
Qwen3.7-Plus	70.5	78.0	75.0	59.9	80.5	75.2	52.1	56.6	67.7
GLM-5.2	72.0	89.2	76.0	63.0	87.0	78.2	62.8	63.0	75.3
DeepSeek-V4-Flash	68.1	76.2	70.1	58.4	81.7	59.1	54.8	61.0	71.0
DeepSeek-V4-Pro	71.4	72.4	79.0	56.5	61.1	71.1	55.2	62.0	72.0
Kimi K2.7 Code	73.6	87.8	79.0	57.3	76.1	72.8	56.9	65.8	69.9
GPT-5.6	76.9	76.0	81.0	68.0	84.2	83.7	84.9	65.3	68.7
Gemini 3.1 Pro	67.5	75.8	83.0	60.1	81.0	77.4	70.7	60.6	74.0
Gemini 3.5 Flash	75.0	88.0	85.0	64.0	74.2	76.2	50.3	63.3	78.0
JIT-Agent + GLM-5.2	78.0	93.9	88.0	69.9	93.3	83.4	83.0	68.4	78.7
JIT-Agent + DeepSeek-V4-Flash	74.0	85.1	82.0	63.8	92.9	83.9	61.3	63.4	73.0

Table 4: Controlled comparison with advanced agent harnesses. For each fixed backbone, we report task performance (Perf.), average token consumption per case in thousands (#Tokens (K)), and API cost per case in USD. Performance is better when higher; tokens and cost are better when lower. The best value within each backbone group is bolded.

Backbone	Harness	DeepSearchQA			xBench-DS			AgentIF		
		Perf. ↑	#Tokens (K)	Cost ↓	Perf. ↑	#Tokens (K)	Cost ↓	Perf. ↑	#Tokens (K)	Cost ↓
DeepSeek-V4-Flash	CLAUDE CODE	79.6	625	\$0.088	75.0	559	\$0.079	66.9	808	\$0.114
	CODEx	77.8	760	\$0.107	70.0	680	\$0.096	58.5	870	\$0.123
	OPeNCoDE	75.9	1,832	\$0.258	65.0	1,157	\$0.159	48.1	950	\$0.135
	HErMES	69.9	1,157	\$0.163	72.0	1,254	\$0.177	60.3	1,000	\$0.142
	NaNoBoT	80.4	924	\$0.131	78.0	527	\$0.075	53.1	1,034	\$0.147
	JIT-Agent	85.1	400	\$0.066	82.0	212	\$0.039	63.8	476	\$0.097
Qwen3.6-Flash	CLAUDE CODE	72.8	710	\$0.140	58.0	650	\$0.128	55.4	900	\$0.177
	CODEx	68.5	980	\$0.193	52.0	874	\$0.172	34.2	839	\$0.170
	OPeNCoDE	64.1	1,968	\$0.384	44.0	1,300	\$0.256	38.7	961	\$0.217
	HErMES	60.8	1,319	\$0.256	36.0	1,155	\$0.227	49.7	1,184	\$0.223
	NaNoBoT	74.2	892	\$0.197	63.0	597	\$0.119	43.5	950	\$0.187
	JIT-Agent	70.3	464	\$0.095	70.0	300	\$0.069	58.3	394	\$0.078

PinchBench; JIT-Agent + DeepSeek-V4-Flash leads DeepPlanning-Shopping at 83.9. The latter pairing also exceeds the stronger DeepSeek-V4-Pro baseline on every reported benchmark, with an average advantage of 8.7 points. DeepPlanning-Travel is the only column not led by a JIT-equipped model, where GLM-5.2 + JIT-Agent reaches 83.0, within 1.9 points of GPT-5.6. Overall, the results support the central claim that improving the operational scaffold can recover a substantial fraction of the capability otherwise sought through backbone scaling.

6.3 Comparison with Advanced Agent Harnesses

Beyond comparing against vanilla backbones, we control the underlying model and vary only the harness. We instantiate each harness on DeepSeek-V4-Flash and Qwen3.6-Flash and evaluate on DeepSearchQA, xBench-DS, and AgentIF. In addition to task performance, we report the average token consumption and API cost per case to separate effective orchestration from gains obtained by simply spending more inference compute.

Performance under backbone control. JIT-Agent achieves the highest performance in four of the six backbone–benchmark settings. With DeepSeek-V4-Flash, it improves over the strongest fixed



Figure 4: Cost-performance trade-offs on DeepSearchQA and AgentIF. Marker color identifies the harness, while filled and open circles distinguish DeepSeek-V4-Flash and Qwen3.6-Flash. The horizontal axis is API cost per case in USD and the vertical axis is task performance. The dark-green step line traces the global Pareto frontier, and the pale yellow-green and blue-green regions contain pairings dominated by at least one Pareto-optimal point.

harness by 4.7 points on DeepSearchQA (85.1 vs. 80.4) and 4.0 points on xBench-DS (82.0 vs. 78.0). With Qwen3.6-Flash, it gains 7.0 points on xBench-DS (70.0 vs. 63.0) and 2.9 points on AgentIF (58.3 vs. 55.4). The two exceptions exhibit a bounded quality-efficiency trade-off: on DeepSeek-V4-Flash AgentIF, JIT-Agent trails Claude Code by 3.1 points, and on Qwen3.6-Flash DeepSearchQA it trails NanoBot by 3.9 points, while using substantially fewer tokens in both cases. No fixed harness dominates across tasks: for example, NanoBot is strongest on Qwen3.6-Flash DeepSearchQA but falls 14.8 points behind JIT-Agent on AgentIF. This variation motivates generating a task-conditioned harness rather than selecting one durable scaffold globally.

Token and cost efficiency. JIT-Agent has the lowest token consumption and API cost in all six controlled settings. Relative to the cheapest fixed harness in each setting, it reduces per-case cost by 14.9–54.1%, with an average reduction of 36.0%. On DeepSeek-V4-Flash xBench-DS, token consumption drops from 527K to 212K and cost from \$0.075 to \$0.039, while performance increases from 78.0 to 82.0. The largest cost reduction occurs on Qwen3.6-Flash AgentIF, where JIT-Agent uses 394K tokens at \$0.078, compared with at least 839K tokens and \$0.170 for the fixed alternatives, yet improves the best fixed-harness score from 55.4 to 58.3. Thus, the gains are not explained by longer trajectories; the generated harnesses typically achieve stronger results with shorter, more selective execution.

6.4 Cost-Performance Pareto Frontiers

Figure 4 visualizes the cost-performance geometry of the controlled comparison on DeepSearchQA and AgentIF. Each point is a backbone-harness pair from Table 4; points closer to the upper-left corner are preferable. On DeepSearchQA, DeepSeek-V4-Flash + JIT-Agent is a strong Pareto-optimal operating point among the evaluated pairings, scoring 85.1 at a per-case cost of \$0.066. Relative to the strongest fixed harness, NanoBot, it gains 4.7 points while reducing cost by 49.6% (\$0.131 → \$0.066). For Qwen3.6-Flash, the generated harness provides a lower-compute operating point: it scores 70.3 at \$0.095, compared with NanoBot’s 74.2 at \$0.197, cutting cost by 51.8% for a 3.9-point trade-off. AgentIF exhibits a different but complementary frontier. With DeepSeek-V4-Flash, JIT-Agent reduces cost by 14.9% relative to Claude Code (\$0.097 vs. \$0.114) while remaining within 3.1 points of

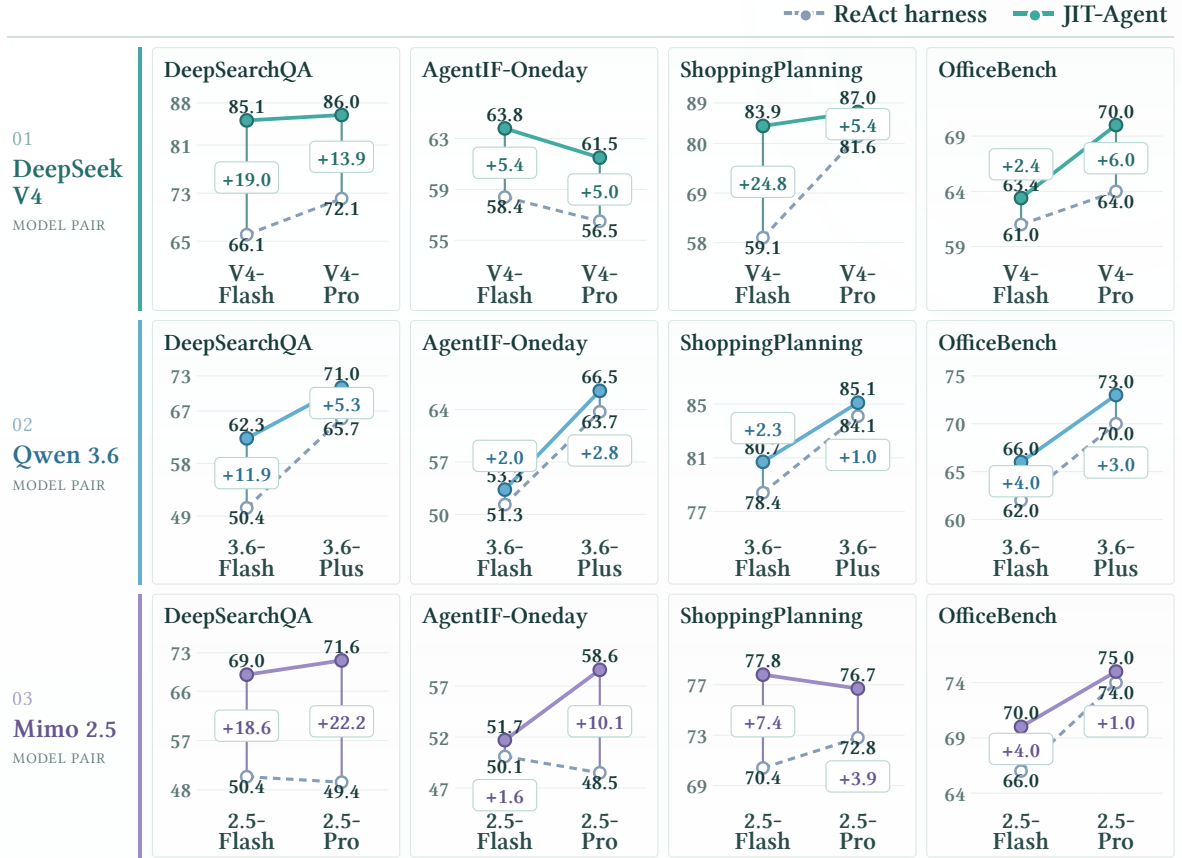


Figure 5: JIT-generated harnesses consistently improve paired backbones over ReAct. Rows group three model families and two variants within each family; columns correspond to DeepSearchQA, AgentIF-Oneday, DeepPlanning-Shopping, and OfficeBench. Each panel compares the same backbone under a fixed ReAct harness (dashed) and a JIT-generated harness (solid), with callouts reporting the absolute score gain. DeepSearchQA uses a 100-example subset; the other three benchmarks use 50-example subsets.

its performance (63.8 vs. 66.9); both therefore lie on the frontier. With Qwen3.6-Flash, JIT-Agent strictly dominates all fixed harnesses, improving the best score from 55.4 to 58.3 while lowering the minimum observed cost from \$0.170 to \$0.078. The frontier is therefore task-dependent, but the consistent leftward shift of the JIT points shows that task-adaptive harnesses improve inference efficiency rather than purchasing accuracy through longer trajectories.

6.5 Generalization across Model Pairs

To examine whether JIT harness generation transfers across model families and variants, we evaluate six backbones spanning three model pairs: DeepSeek-V4-Flash/Pro, Qwen3.6-Flash/Plus, and MIMO-V2.5-Flash/Pro. For each backbone, we hold the model fixed and compare its standard ReAct harness with the JIT-generated harness on DeepSearchQA, AgentIF-Oneday, DeepPlanning-Shopping, and OfficeBench.

Across all 24 directly matched backbone–benchmark comparisons, the JIT-generated harness outperforms ReAct, with an average gain of 7.6 points. The effect holds for every family and both variants in each pair: the average improvement is 10.2 points for DeepSeek V4, 4.0 for Qwen 3.6, and 8.6 for MIMO 2.5. DeepSearchQA shows the largest average gain at 15.2 points, including +22.2 for MIMO-V2.5-Pro and +19.0 for DeepSeek-V4-Flash; DeepPlanning-Shopping rises by 7.5 points on average, including a +24.8 gain for DeepSeek-V4-Flash. These consistent within-backbone gains show

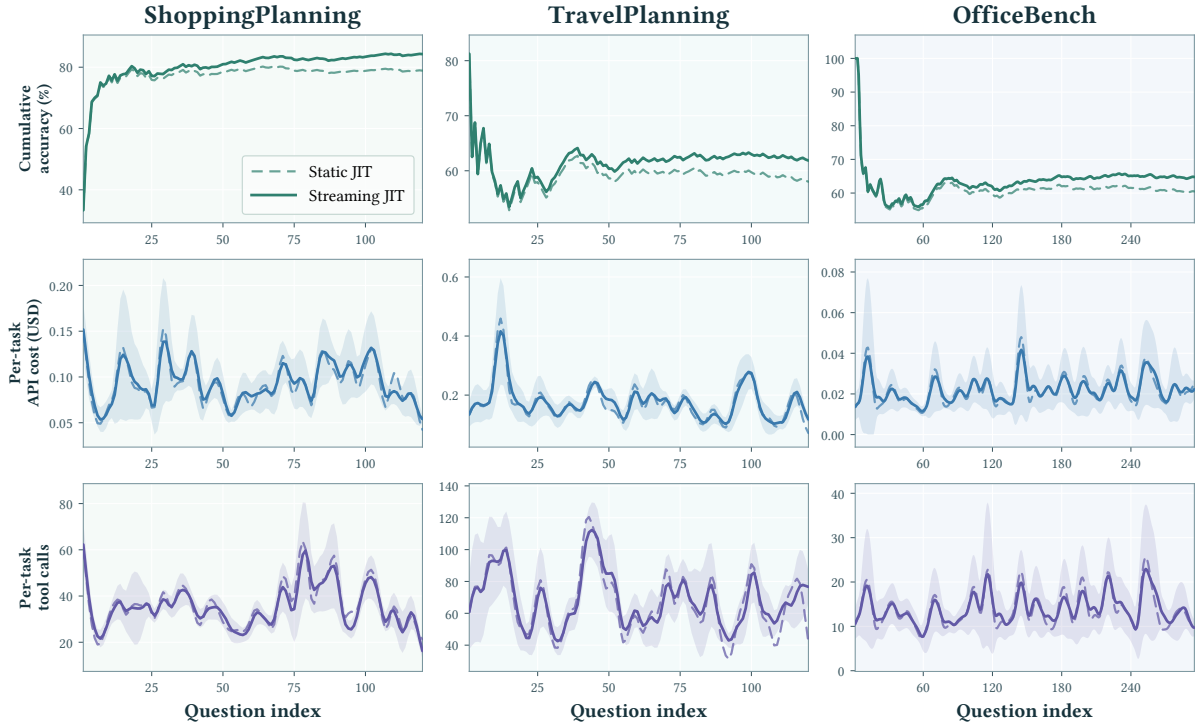


Figure 6: Streaming test-time harness evolution across task streams. Cumulative accuracy (top), per-task API cost (middle), and per-task tool calls (bottom) on DeepPlanning-Shopping, DeepPlanning-Travel, and OfficeBench. Dashed curves represent Static JIT, where task-specific harness generations are independent, while solid curves represent Streaming JIT, which continuously incorporates execution feedback as new tasks arrive. Shaded regions show local variation around the streaming trajectories. Streaming JIT finishes with higher cumulative accuracy on all three benchmarks, while API-cost and tool-use trajectories remain task-dependent and of broadly similar scale.

that harness intelligence transfers across model families and variants rather than compensating for a particular backbone.

6.6 Test-Time Harness Evolution

JIT-Agent is designed not only to generate a harness once, but also to improve its archive from execution feedback. We compare *Static JIT*, where harness generations are independent, with *Streaming JIT*, which retrieves and updates harnesses throughout the evaluation stream.

Streaming JIT finishes above the static variant on DeepPlanning-Shopping, DeepPlanning-Travel, and OfficeBench. Across all three streams, the cumulative-accuracy advantage emerges as execution feedback accumulates and remains positive through the end of evaluation. The accompanying cost and tool-call traces suggest that the endpoint gains are not uniformly coupled to larger interaction budgets. This pattern is consistent with the objective of Evo-GDPO, which retains harnesses only when they advance the archive frontier.

6.7 Visualization of Generated Harnesses

The quantitative results establish that generated harnesses improve execution; we next inspect what JIT-Agent actually generates. Figures 7 and 8 visualize two harnesses produced under the same four-module protocol for tasks with sharply different computational structure. Their differences extend beyond prompts or module names: the generated planning representation, action topology, persistent state, and exposed capabilities are reorganized around the demands of each task.

Palimpsest — graph-planned artifact execution

A task becomes a dependency graph, executes with bounded parallelism, and leaves reusable state.

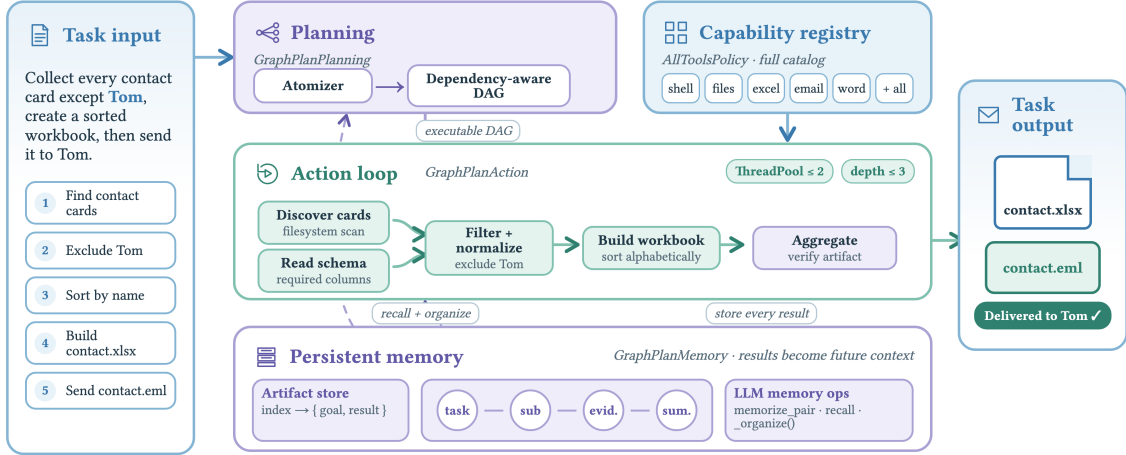


Figure 7: Palimpsest: graph-planned artifact execution. GraphPlanPlanning converts the contact-processing request into a DAG; GraphPlanAction executes it with bounded width and depth, while GraphPlanMemory stores reusable artifacts and reasoning state.

Artifact production as a dependency graph. PALIMPSEST is generated for a cross-application task that must discover contact cards, exclude one person, normalize and sort the remaining records, create a workbook, and deliver it by email. The harness compiles these requirements into an explicit DAG: discovery and schema inspection provide prerequisites for filtering, workbook construction waits for normalized records, and delivery waits for artifact verification. GraphPlanAction then executes ready nodes with bounded width and depth, while GraphPlanMemory indexes intermediate goals, evidence, and artifacts so that downstream nodes consume completed results rather than reconstructing them from the transcript. The resulting harness matches the task’s artifact dependencies and commit order, turning a long natural-language request into a verifiable production pipeline.

Deep research as bounded recursive delegation. TRAPDOOR addresses a multi-hop identity question whose clues span a historical quotation, a university merger, a memoir, and a research paper. Here a fixed DAG would commit too early to an uncertain evidence path, so the generated harness uses DynamicDecomposer to form and revise sub-questions and augments the research tool policy with a synthetic delegate capability. A delegate call opens a private research subagent with its own memory, research-only tools, and a five-step budget; the returned answer re-enters the parent loop as an ordinary observation. In parallel, FactGraphMemory extracts compact key–value facts from the growing history, allowing the parent to integrate evidence across branches without inheriting every subagent trace. Thus, the harness makes recursive research a runtime primitive specifically for a task whose uncertainty is resolved through branching evidence collection.

The contrast is the central qualitative result: the same generator maps one task to graph execution with an artifact store and another to recursive orchestration with a fact store. The shared protocol constrains interfaces, not behavior. The additional cases in Section A broaden this pattern to hierarchical context folding, completion-gated tool access, phase-conditioned execution, evidence-matrix search, selective context rendering, typed computational state, deterministic per-file verification, and local action repair; their flows are shown in Figures A.1 to A.8.

Trapdoor — delegation as a runtime primitive

A synthetic tool call opens a bounded sub-agent; its answer returns through the ordinary observation loop.

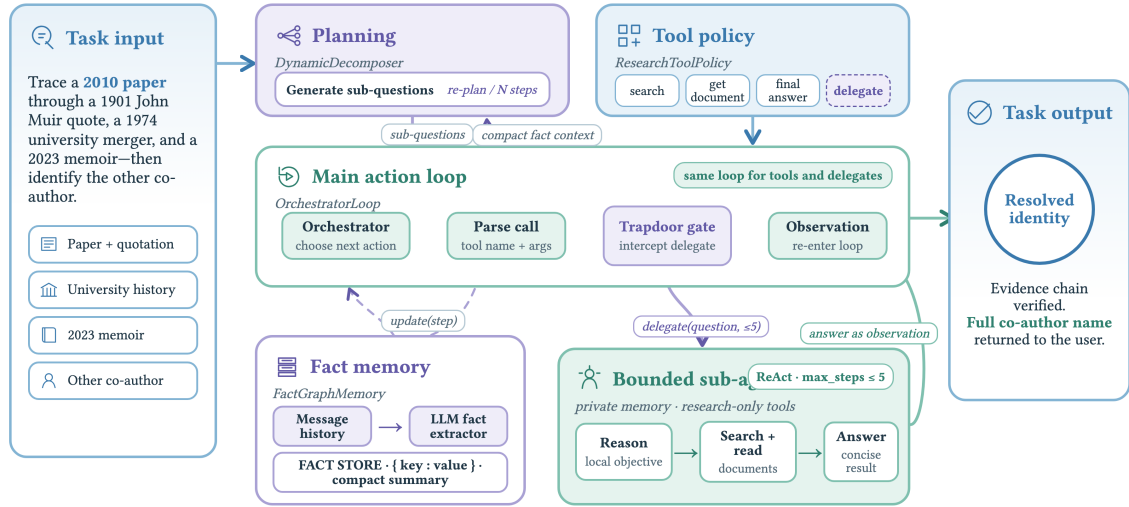


Figure 8: Trapdoor: bounded research behind a delegated tool call. A synthesized `delegate` capability is intercepted by `OrchestratorLoop`, which runs a bounded subagent and writes extracted facts into `FactGraphMemory`.

7 Conclusion and Future Work

Conclusion. We introduced JIT-Agent, a dedicated model for synthesizing, repairing, and evolving task-conditioned agent harnesses at inference time. By expressing the harness through a composable four-module protocol and training the generator through customization learning, repair supervision, and Evo-GDPO, JIT-Agent turns harness construction from a manually engineered, ahead-of-time artifact into a learned capability. Across deep research, daily work, planning, and workspace tasks, JIT-generated harnesses consistently strengthen their underlying backbones, compete with frontier models and advanced fixed harnesses, and improve cost–performance trade-offs. These results establish harness intelligence, *i.e.*, the capacity to adapt the operational scaffold through which a model acts, as a trainable and transferable source of agent capability beyond model weights alone.

Future Work. We view harness intelligence as another fundamental dimension of scaling, alongside model capacity and inference compute, and JIT-Agent as one step toward realizing it. The longer-term opportunity is a model–harness co-design paradigm in which foundation models are trained together with the operational structures that shape their memory, planning, action, and tool use. Nevertheless, we believe future systems need not follow the deliberately radical form explored in this work, where the entire scaffold can be redesigned just in time: production runtimes may instead retain a stable core while allowing the model to construct, revise, or replace selected harness components when the task demands it. As this paradigm matures, the capabilities developed here for harness synthesis, repair, and evolution will increasingly be internalized by foundation models themselves. Models will then learn not only to act within a given harness, but also to improve the harness through which they act, opening a broad research agenda around adaptive interfaces, verifiable runtime modification, and jointly scaling models with their execution systems.

Contributors

Project Lead Guibin Zhang

Core Contributor Guibin Zhang, Leo Lu, Fangzhou Xie

Contributor Kang Zhu, Junhao Wang, Zhifei Xie, Zhaochen Yu, Zihang Liu, Zhongxiang Sun, Qiankun Li, Yue Liao, Heng Chang, Xiaobin Hu, Qibing Ren

Corresponding Wangchunshu Zhou, Shuicheng Yan

References

Salaheddin Alzu'bi, Baran Nama, Arda Kaz, Anushri Eswaran, Weiyuan Chen, Sarvesh Khetan, Rishab Bala, Tu Vu, and Sewoong Oh. Roma: Recursive open meta-agent framework for long-horizon multi-agent systems, 2026. URL <https://arxiv.org/abs/2602.01848>.

Anthropics. GitHub - anthropics/claude-code: Claude Code is an agentic coding tool that lives in your terminal, understands your codebase, and helps you code faster by executing routine tasks, explaining complex code, and handling git workflows - all through natural language commands. — github.com. <https://github.com/anthropics/claude-code>, 2025. [Accessed 04-01-2026].

Apodex Team. Apodex-1.0: Self-evolving heavy-duty solver. <https://www.apodex.com/blog/apodex-1.0>, 2026.

Fei Bai, Huatong Song, Shuang Sun, Daixuan Cheng, Yike Yang, Chuan Hao, Renyuan Li, Feng Chang, Yuan Wei, Ran Tao, Bryan Dai, Jian Yang, Wayne Xin Zhao, and Ji-Rong Wen. Clawgym: A scalable framework for building effective claw agents, 2026. URL <https://arxiv.org/abs/2604.26904>.

James Burgess, Jan N. Hansen, Duo Peng, Yuhui Zhang, Alejandro Lozano, Min Woo Sun, Emma Lundberg, and Serena Yeung-Levy. Papersearchqa: Learning to search and reason over scientific papers with rlvr, 2026. URL <https://arxiv.org/abs/2601.18207>.

Pengfei Cao, Tianyi Men, Wencan Liu, Jingwen Zhang, Xuzhao Li, Xixun Lin, Dianbo Sui, Yanan Cao, Kang Liu, and Jun Zhao. Large language models for planning: A comprehensive and systematic survey, 2025. URL <https://arxiv.org/abs/2505.19683>.

Angelica Chen, David Dohan, and David So. Evoprompting: Language models for code-level neural architecture search. *Advances in neural information processing systems*, 36:7787–7817, 2023.

Kaiyuan Chen, Yixin Ren, Yang Liu, Xiaobo Hu, Haotong Tian, Tianbao Xie, Fangfu Liu, Haoye Zhang, Hongzhang Liu, Yuan Gong, Chen Sun, Han Hou, Hui Yang, James Pan, Jianan Lou, Jiayi Mao, Jizheng Liu, Jinpeng Li, Kangyi Liu, Kenkun Liu, Rui Wang, Run Li, Tong Niu, Wenlong Zhang, Wenqi Yan, Xuanzheng Wang, Yuchen Zhang, Yi-Hsin Hung, Yuan Jiang, Zexuan Liu, Zihan Yin, Zijian Ma, and Zhiwen Mo. xbench: Tracking agents productivity scaling with profession-aligned real-world evaluations, 2025a. URL <https://arxiv.org/abs/2506.13651>.

Kaiyuan Chen, Qimin Wu, Taiyu Hou, Tianhao Tang, Xueyu Hu, Yuchen Hou, Bikun Li, Chengming Qian, Guoyin Wang, Haolin Chen, Haotong Tian, Haoye Zhang, Haoyu Bian, Hongbing Pan, Hongkang Zhang, Hongyi Zhou, Jiaqi Cai, Jiewu Rao, Jiyan Ren, Keduan Huang, Lucia Zhu Huang, Mingyu Yuan, Naixu Guo, Qicheng Tang, Qinyan Zhang, Shuai Chen, Siheng Chen, Ting Ting Li, Xiaoxing Guo, Yaocheng Zuo, Yaoqi Guo, Yinan Wang, Yinzhou Yu, Yize Wang, Yuan Jiang, Yuan Tian, Yuanshuo Zhang, Yuxuan Liu, Yvette Yan Zeng, Zenyu Shan, Zihan Yin, Xiaobo Hu, Yang Liu, Yixin Ren, and Yuan Gong. Agentif-oneday: A task-level instruction-following benchmark for general ai agents in daily scenarios, 2026a. URL <https://arxiv.org/abs/2601.20613>.

Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, and Jian Luan. Harnessx: A composable, adaptive, and evolvable agent harness foundry, 2026b. URL <https://arxiv.org/abs/2606.14249>.

- Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, Sahel Sharifmoghaddam, Yanxi Li, Haoran Hong, Xinyu Shi, Xuye Liu, Nandan Thakur, Crystina Zhang, Luyu Gao, Wenhua Chen, and Jimmy Lin. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent, 2025b. URL <https://arxiv.org/abs/2508.06600>.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
- ClaudeCode. Agent Skills - Claude Code Docs — code.claude.com. <https://code.claude.com/docs/en/skills>, 2025. [Accessed 03-01-2026].
- DeepSeek-AI. Deepseek-v4 preview release. <https://api-docs.deepseek.com/news/news260424/>, 2026.
- Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. Plan-and-act: Improving planning of agents for long-horizon tasks. *arXiv preprint arXiv:2503.09572*, 2025a.
- Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. Plan-and-act: Improving planning of agents for long-horizon tasks, 2025b. URL <https://arxiv.org/abs/2503.09572>.
- Google DeepMind. Gemini 3.1 pro model card. <https://deepmind.google/models/model-cards/gemini-3-1-pro>, 2026a.
- Google DeepMind. Gemini 3.5 flash model card. <https://deepmind.google/models/model-cards/gemini-3-5-flash/>, 2026b.
- Nikita Gupta, Riju Chatterjee, Lukas Haas, Connie Tao, Andrew Wang, Chang Liu, Hidekazu Oiwa, Elena Gribovskaya, Jan Ackermann, John Blitzer, Sasha Goldshtein, and Dipanjan Das. Deepsearchqa: Bridging the comprehensiveness gap for deep research agents, 2026. URL <https://arxiv.org/abs/2601.20975>.
- HKUDS. Nanobot: Lightweight open-source ai agent. <https://github.com/HKUDS/nanobot>, 2026.
- Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, and Yafeng Deng. Evermemos: A self-organizing memory operating system for structured long-horizon reasoning, 2026a. URL <https://arxiv.org/abs/2601.02163>.
- Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language model, 2024a. URL <https://arxiv.org/abs/2408.09559>.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024b.
- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. Memory in the age of ai agents, 2025. URL <https://arxiv.org/abs/2512.13564>.

- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. Memory in the age of ai agents, 2026b. URL <https://arxiv.org/abs/2512.13564>.
- Jiajie Jin, Xiaoxi Li, Guanting Dong, Yuyao Zhang, Yutao Zhu, Yang Zhao, Hongjin Qian, and Zhicheng Dou. Hira: A hierarchical reasoning framework for decoupled planning and execution in deep search, 2025. URL <https://arxiv.org/abs/2507.02652>.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- LangGraph. GitHub - langchain-ai/langgraph: Build resilient language agents as graphs. — github.com. <https://github.com/langchain-ai/langgraph>, 2025. [Accessed 03-01-2026].
- Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, and Yujin Tang. Recursive harness self-improvement, 2026a. URL <https://arxiv.org/abs/2607.15524>.
- Sanghoon Lee, Jiyeong Chae, and Kyung-Joon Park. Harness engineering for physical ai: Robot middleware is the harness layer, 2026b. URL <https://arxiv.org/abs/2606.09416>.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026c. URL <https://arxiv.org/abs/2603.28052>.
- Yoonsang Lee, Howard Yen, Xi Ye, and Danqi Chen. Agentic aggregation for parallel scaling of long-horizon agentic tasks, 2026d. URL <https://arxiv.org/abs/2604.11753>.
- Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale, 2026a. URL <https://arxiv.org/abs/2603.02176>.
- Xiaoxi Li, Wenxiang Jiao, Jiarui Jin, Guanting Dong, Jiajie Jin, YINUO Wang, Hao Wang, Yutao Zhu, Ji-Rong Wen, Yuan Lu, and Zhicheng Dou. Deepagent: A general reasoning agent with scalable toolsets, 2026b. URL <https://arxiv.org/abs/2510.21618>.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026. URL <https://arxiv.org/abs/2604.25850>.
- Jiaxi Liu, Yanzuo Jiang, Guibin Zhang, Zihan Zhang, Heng Chang, Zhenfei Yin, Qibing Ren, and Junchi Yan. Todoevolve: Learning to architect agent planning systems, 2026a. URL <https://arxiv.org/abs/2602.07839>.
- Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Peter Belcak, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, Yejin Choi, Jan Kautz, and Pavlo Molchanov. Gdpo: Group reward-decoupled normalization policy optimization for multi-reward rl optimization, 2026b. URL <https://arxiv.org/abs/2601.05242>.

- Zewen Liu, Zhan Shi, Yisi Sang, Bing He, Minhua Lin, Tianxin Wei, Dakuo Wang, Benoit Dumoulin, Wei Jin, and Hanqing Lu. Adaptive auto-harness: Sustained self-improvement for agentic system deployment on open-ended task streams, 2026c. URL <https://arxiv.org/abs/2606.01770>.
- Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. AutoHarness: Improving llm agents by automatically synthesizing a code harness, 2026. URL <https://arxiv.org/abs/2603.03329>.
- Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyang Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, Philip S. Yu, and Ming Zhang. Large language model agent: A survey on methodology, applications and challenges, 2025. URL <https://arxiv.org/abs/2503.21460>.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023.
- Qianyu Meng, Yanan Wang, Liyi Chen, Qimeng Wang, Chengqiang Lu, Wei Wu, Yan Gao, Yi Wu, and Yao Hu. Agent harness for large language model agents: A survey. *Preprints*, April 2026a. doi: 10.20944/preprints202604.0428.v2. URL <https://doi.org/10.20944/preprints202604.0428.v2>.
- Qianyu Meng, Yanan Wang, Liyi Chen, Qimeng Wang, Chengqiang Lu, Wei Wu, Yan Gao, Yi Wu, and Yao Hu. Agent harness for large language model agents: A survey. 2026b.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.
- Microsoft. GitHub - ag2ai/ag2: AG2 (formerly AutoGen): The Open-Source AgentOS. <https://github.com/ag2ai/ag2>, 2024. [Accessed 31-08-2025].
- Moonshot AI. Kimi k2.7 code: Open-source agentic coding model. <https://www.kimi.com/resources/kimi-k2-7-code>, 2026.
- Jun Nie, Yonggang Zhang, Qianshu Cai, Yiu-ming Cheung, Xinmei Tian, and Bo Han. EvolveNet: Collaborative harness evolution for agent self-improvement, 2026a. URL <https://arxiv.org/abs/2608.04968>.
- Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Yike Guo, Xinmei Tian, and Bo Han. TTHE: Test-time harness evolution, 2026b. URL <https://arxiv.org/abs/2607.08124>.

- Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, et al. Code as agent harness. *arXiv preprint arXiv:2605.18747*, 2026.
- Nous Research. Hermes agent. <https://github.com/NousResearch/hermes-agent>, 2026.
- OpenAI. Codex | AI Coding Partner from OpenAI — openai.com. <https://openai.com/codex/>, 2025.
- OpenAI. Gpt-5.6: Frontier intelligence that scales with your ambition. <https://openai.com/index/gpt-5-6/>, 2026.
- OpenClaw Contributors. Openclaw: Your own personal ai assistant. <https://github.com/openclaw/openclaw>, 2026.
- OpenCode-AI. GitHub - opencode-ai/opencode: A powerful AI coding agent. Built for the terminal. — github.com. <https://github.com/opencode-ai/opencode>, 2025.
- Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-language agent harnesses. *arXiv preprint arXiv:2603.25723*, 2026a.
- Wenbo Pan, Shujie Liu, Chin-Yew Lin, Jingying Zeng, Xianfeng Tang, Xiangyang Zhou, Yan Lu, and Xiaohua Jia. Evolving agents in the dark: Retrospective harness optimization via self-preference, 2026b. URL <https://arxiv.org/abs/2606.05922>.
- PinchBench. Best Models by Success Rate | PinchBench — pinchbench.com. <https://pinchbench.com/>, 2026. [Accessed 11-04-2026].
- Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.
- Hongjin Qian, Zhao Cao, and Zheng Liu. Memobrain: Executive memory as an agentic brain for reasoning, 2026. URL <https://arxiv.org/abs/2601.08079>.
- Tianrui Qin, Qianben Chen, Sinuo Wang, He Xing, King Zhu, He Zhu, Dingfeng Shi, Xinxin Liu, Ge Zhang, Jiaheng Liu, Yuchen Eleanor Jiang, Xitong Gao, and Wangchunshu Zhou. Flash-searcher: Fast and effective web agents via dag-based parallel execution, 2025. URL <https://arxiv.org/abs/2509.25301>.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. URL <https://arxiv.org/abs/2307.16789>.
- Qwen Team. Qwen3.7-plus: Multimodal agent intelligence. <https://qwen.ai/home>, 2026.
- Jianhao Ruan, Zhihao Xu, Yiran Peng, Fashen Ren, Zhaoyang Yu, Xinbing Liang, Jinyu Xiang, Yongru Chen, Bang Liu, Chenglin Wu, Yuyu Luo, and Jiayi Zhang. Aorchestra: Automating sub-agent creation for agentic orchestration, 2026. URL <https://arxiv.org/abs/2602.03786>.
- Anjana Sarkar and Soumyendu Sarkar. Survey of llm agent communication with mcp: A software design pattern centric review, 2025. URL <https://arxiv.org/abs/2506.05364>.
- Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. Agentsquare: Automatic llm agent search in modular design space. *arXiv preprint arXiv:2410.06153*, 2024.
- Shuai Shao, Kangning Zhang, Qingyao Li, Shijian Wang, Hao Wang, Wenxiang Jiao, Yuan Lu, Yi Guo, Weiwen Liu, and Weinan Zhang. Harness-R1: Learning to edit executable runtime harnesses from agent failure trajectories, 2026. URL <https://arxiv.org/abs/2608.02276>.

- Dingfeng Shi, Jingyi Cao, Qianben Chen, Weichen Sun, Weizhen Li, Hongxuan Lu, Fangchen Dong, Tianrui Qin, King Zhu, Minghao Liu, Jian Yang, Ge Zhang, Jiaheng Liu, Changwang Zhang, Jun Wang, Yuchen Eleanor Jiang, and Wangchunshu Zhou. Taskcraft: Automated generation of agentic tasks, 2025. URL <https://arxiv.org/abs/2506.10055>.
- Yifan Shi, Wei Zhang, and Tianyi Cui. A programming paradigm for spatiotemporal composability, 2026. URL <https://github.com/cordiverse/paper>. Preprint, draft of August 13, 2026.
- Noah Shinn, Beck Labash, and Ashwin Gopinath. Reflexion: an autonomous agent with dynamic memory and self-reflection. *arXiv preprint*, abs/2303.11366, 2023. doi: 10.48550/arXiv.2303.11366. URL <https://doi.org/10.48550/arXiv.2303.11366>.
- Xiaoshuai Song, Haofei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. Envscaler: Scaling tool-interactive environments for llm agent via programmatic synthesis, 2026. URL <https://arxiv.org/abs/2601.05808>.
- Haotian Sun, Yuchen Zhuang, Ling kai Kong, Bo Dai, and Chao Zhang. Adapllanner: Adaptive planning from feedback with language models. *Advances in neural information processing systems*, 36:58202–58245, 2023.
- Hongru Wang, Cheng Qian, Wan jun Zhong, Xiushi Chen, Jiahao Qiu, Shijue Huang, Bowen Jin, Mengdi Wang, Kam-Fai Wong, and Heng Ji. Otc: Optimal tool calls via reinforcement learning. *arXiv preprint arXiv:2504.14870*, 2025a.
- Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, and Saravan Rajmohan. Odysseybench: Evaluating llm agents on long-horizon complex office application workflows, 2025b. URL <https://arxiv.org/abs/2508.09124>.
- Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. Officebench: Benchmarking language agents across multiple applications for office automation, 2024. URL <https://arxiv.org/abs/2407.19056>.
- Jialong Wu, Wenbiao Yin, Yong Jiang, Zhenglin Wang, Zekun Xi, Runnan Fang, Linhai Zhang, Yulan He, Deyu Zhou, Pengjun Xie, and Fei Huang. Webwalker: Benchmarking llms in web traversal, 2025. URL <https://arxiv.org/abs/2501.07572>.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework, August 01, 2023 2023.
- Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, Minhao Cheng, Shuai Wang, Hong Cheng, and Jingren Zhou. Resum: Unlocking long-horizon search intelligence via context summarization, 2026. URL <https://arxiv.org/abs/2509.13313>.
- Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward, 2026. URL <https://arxiv.org/abs/2602.12430>.
- B. Y. Yan, Chaofan Li, Hongjin Qian, Shuqi Lu, and Zheng Liu. General agentic memory via deep research, 2025. URL <https://arxiv.org/abs/2511.18423>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.

- Xiaofang Yang, Lijun Li, Heng Zhou, Tong Zhu, Xiaoye Qu, Yuchen Fan, Qianshan Wei, Rui Ye, Li Kang, Yiran Qin, Zhiqiang Kou, Daizong Liu, Qi Li, Ning Ding, Siheng Chen, and Jing Shao. Toward efficient agents: Memory, tool learning, and planning, 2026. URL <https://arxiv.org/abs/2601.14192>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023a. URL <https://arxiv.org/abs/2210.03629>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023b.
- Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Zile Qiao, Xinyu Wang, Pengjun Xie, Fei Huang, Siheng Chen, Jingren Zhou, and Yong Jiang. Agentfold: Long-horizon web agents with proactive context management, 2025. URL <https://arxiv.org/abs/2510.24699>.
- Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, and Deqing Yang. Evoagent: Towards automatic multi-agent generation via evolutionary algorithms. *arXiv preprint arXiv:2406.14228*, 2024.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic "differentiation" via text, 2024. URL <https://arxiv.org/abs/2406.07496>.
- Z.ai. Glm-5.2: Built for long-horizon tasks. <https://z.ai/blog/glm-5.2>, 2026.
- Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models, 2026a. URL <https://arxiv.org/abs/2512.24601>.
- Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent architecture search via agentic supernet. *arXiv preprint arXiv:2502.04180*, 2025a.
- Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. Memevolve: Meta-evolution of agent memory systems, 2025b. URL <https://arxiv.org/abs/2512.18746>.
- Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves, 2026b. URL <https://arxiv.org/abs/2606.09498>.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating Agentic Workflow Generation, October 2024. URL <http://arxiv.org/abs/2410.10762>. arXiv:2410.10762.
- Luan Zhang, Ruochen Zhou, Dandan Song, Zhengyu Chen, Yuhang Tian, Jun Yang, Huipeng Ma, Chenhao Li, Guangyuan Feng, Xudong Li, Yizhou Jin, and Yan Xu. HarnessCompass: Guiding automatic harness evolution toward generalizable and effective agent harnesses, 2026c. URL <https://arxiv.org/abs/2608.01918>.
- Yinger Zhang, Shutong Jiang, Renhao Li, Jianhong Tu, Yang Su, Lianghao Deng, Xudong Guo, Chenxu Lv, and Junyang Lin. Deepplanning: Benchmarking long-horizon agentic planning with verifiable constraints, 2026d. URL <https://arxiv.org/abs/2601.18137>.

- Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Chunyang Zhu, Jiaxing Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, Zhengru Fang, Yi Nie, Changxu Wei, Yu Wang, Wenbo Ding, and Chao Yu. Harness vla: Steering frozen vlas into reliable manipulation primitives via memory-guided agents, 2026e. URL <https://arxiv.org/abs/2607.08448>.
- Yuxiang Zhang, Jiangming Shu, Ye Ma, Xueyuan Lin, Shangxi Wu, and Jitao Sang. Memory as action: Autonomous context curation for long-horizon agentic tasks, 2025c. URL <https://arxiv.org/abs/2510.12635>.
- Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. Skillweaver: Web agents can self-improve by discovering and honing skills, 2025. URL <https://arxiv.org/abs/2504.07079>.
- Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, Jianghao Lin, and Weinan Zhang. Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering, 2026. URL <https://arxiv.org/abs/2604.08224>.
- Tailin Zhou. Hierarchical self-improvement: A framework for task-specific evolvable agent harnesses, 2026. URL <https://arxiv.org/abs/2608.08466>.
- He Zhu, Tianrui Qin, King Zhu, Heyuan Huang, Yeyi Guan, Jinxiang Xia, Yi Yao, Hanhao Li, Ningning Wang, Pai Liu, Tianhao Peng, Xin Gui, Xiaowan Li, Yuhui Liu, Yuchen Eleanor Jiang, Jun Wang, Changwang Zhang, Xiangru Tang, Ge Zhang, Jian Yang, Minghao Liu, Xitong Gao, Jiaheng Liu, and Wangchunshu Zhou. Oagents: An empirical study of building effective agents, 2025. URL <https://arxiv.org/abs/2506.15741>.

A Additional Generated Harness Visualizations

This appendix complements the two representative cases in Section 6.7 with eight additional harnesses generated under the same four-module protocol. The examples span constraint-heavy shopping and travel, phased web production, clue-driven research, evidence-sensitive retrieval, numerical analysis, and workspace manipulation. They are not variants of a single execution template: JIT-Agent selects a state representation and control regime suited to the structure of each request, then instantiates planning, action, memory, and capability orchestration around that choice. Collectively, the cases make task-conditioned harness customization visible at the level of execution semantics rather than prompt wording.

JIT HARNESS ATLAS · 03

Origami — hierarchical work, folded context

Independent subtask memories preserve the work; a deliberate fold replaces only the active context.

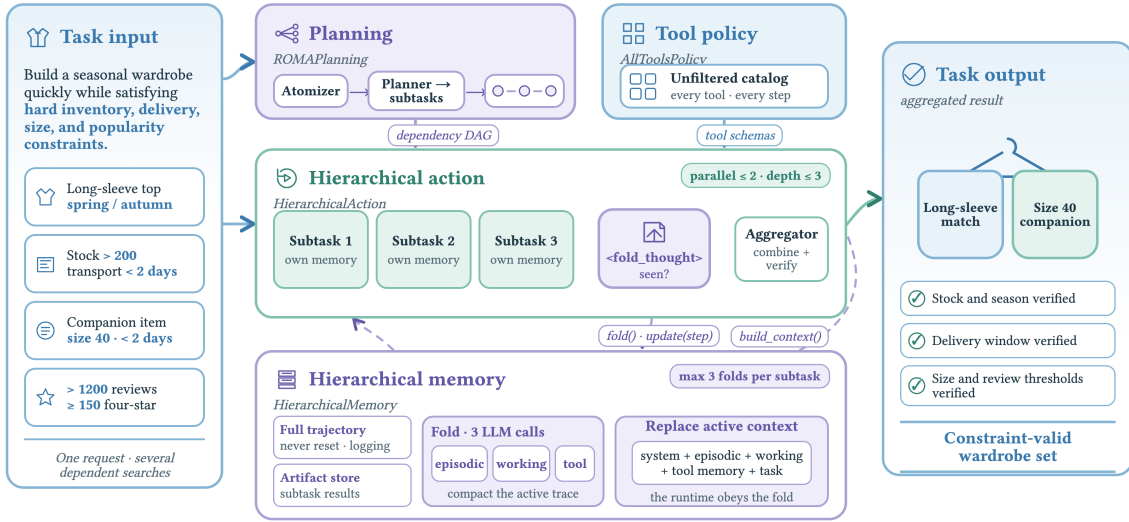


Figure A.1: Origami: hierarchical work with folded context. ROMAPPlanning creates isolated subtasks; HierarchicalMemory retains their trajectories and artifacts while `fold_thought` replaces only the active working context before aggregation.

Figures A.1 and A.2 illustrate two task-specific ways to control long-horizon execution through memory. ORIGAMI must assemble a seasonal wardrobe under coupled stock, delivery, size, review-count, and rating constraints. ROMAPPlanning decomposes the request into dependent product searches, HierarchicalAction runs at most two branches in parallel to bounded depth, and each branch retains its own trajectory and artifacts. When a branch grows long, `fold_thought` replaces only its active working context; completed subtask results remain available for the final constraint check. The harness therefore spends context on the unresolved portion of a combinatorial shopping task without discarding earlier product evidence.

TURNSTILE, by contrast, serves a travel request that specifies dates, passengers, rooms, a departure window, a highest-rated hotel, attractions, meals, and daily routes. The generated TravelPlanning module compiles these obligations into a typed `travel_spec` and collection checklist, and DataStoreMemory assigns a dedicated bucket to each evidence class. DynamicToolPolicy exposes search tools for the missing bucket but withholds `final_answer` until `is_complete()` succeeds. Here memory is not used primarily to compress a long trace; it acts as an executable coverage contract that prevents itinerary synthesis before the required travel facts have been collected.

Figures A.3 and A.4 show how explicit state can continuously reshape later decisions. GEARBOX is

Turnstile — evidence first, itinerary second

A task-specific checklist withholds final_answer until every required travel fact has been collected.

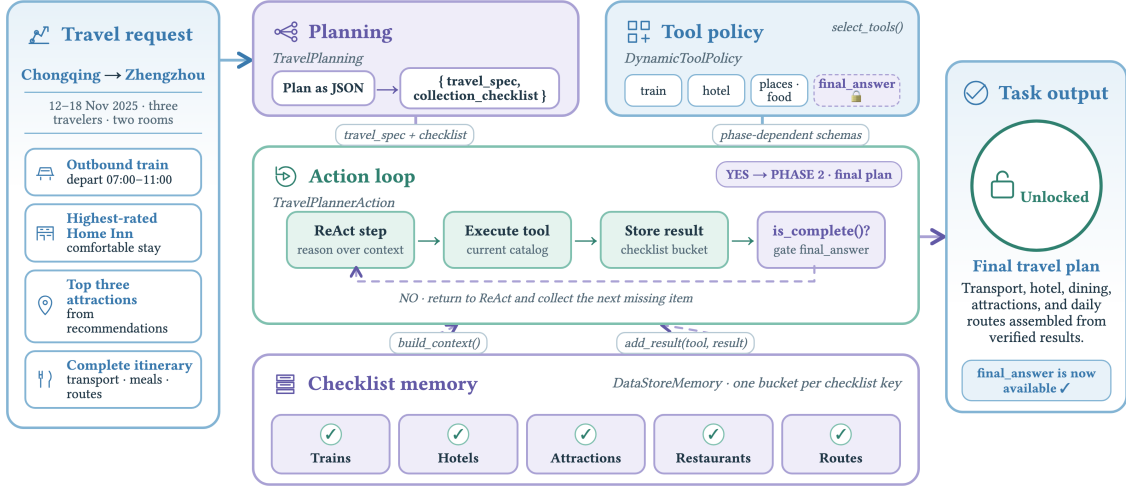


Figure A.2: Turnstile: evidence first, itinerary second. TravelPlanning emits a travel specification and checklist, DataStoreMemory tracks the required evidence buckets, and DynamicToolPolicy exposes final_answer only after is_complete() succeeds.

Gearbox — one phase shift rewrites the agent

The shared register switches tools and memory schemas across search → collect → build.

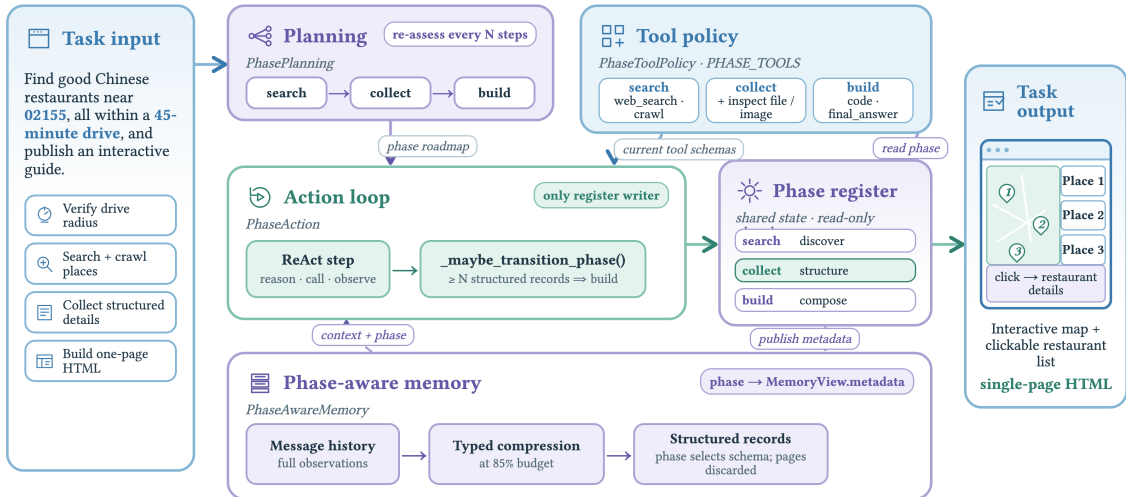


Figure A.3: Gearbox: one phase shift rewrites the agent. PhaseAction is the sole writer of a shared phase register; PhaseToolPolicy and PhaseAwareMemory read that state to switch both exposed capabilities and typed memory schemas.

Pegboard — research as an evidence matrix

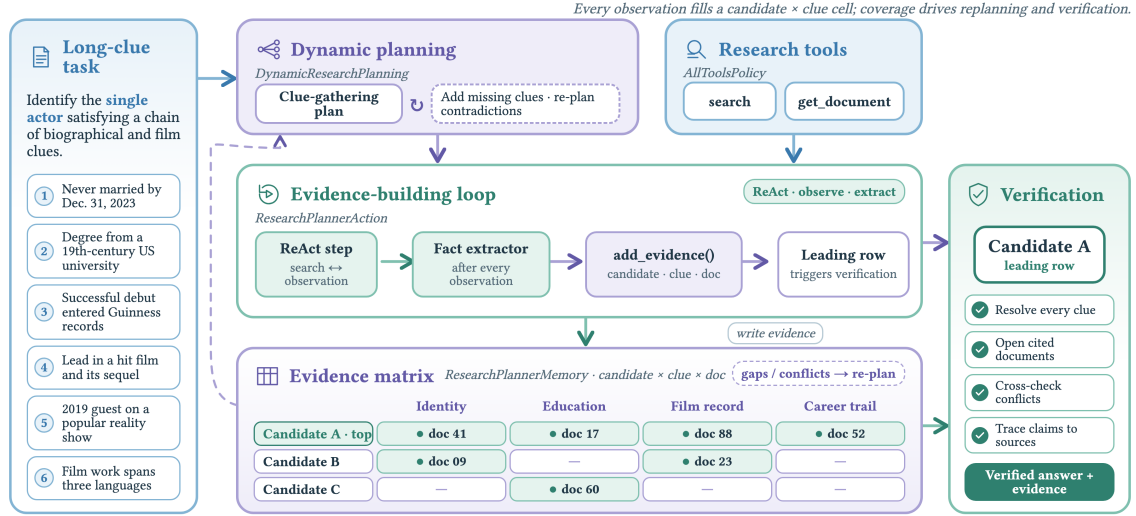


Figure A.4: Pegboard: research as an evidence matrix. Every observation is extracted into a candidate × clue cell with a document identifier; matrix coverage drives both `DynamicResearchPlanning` and the transition to source-grounded verification.

generated to find Chinese restaurants within a specified driving radius and publish the results as an interactive one-page guide. Its `PhaseAction` is the sole writer of a shared *search–collect–build* register; the transition to construction occurs only after enough structured restaurant records have been gathered. Both `PhaseToolPolicy` and `PhaseAwareMemory` read that register, switching from web discovery, to record inspection, to code and publication while compressing observations into the schema needed by the current phase. The model thus creates a phase machine because the task changes modality from evidence acquisition to artifact construction.

PEGBOARD instead tackles an identification problem defined by six biographical and film clues. It represents progress as a candidate × clue matrix in which every extracted claim retains its document identifier. Empty cells and contradictions drive `DynamicResearchPlanning` toward targeted searches, while a sufficiently supported row triggers a separate verification phase that reopens cited documents and resolves conflicts. Unlike a generic research transcript, this state makes coverage and source provenance directly actionable. The contrast with GEARBOX shows that generated state may be a small phase variable or a task-shaped evidence structure, depending on what must govern the next action.

Figures A.5 and A.6 customize what subsequent reasoning receives from prior execution. APPRAISER answers a classical materia-medica identification question by reconciling botanical traits with textual evidence. After every search or reading step, an auxiliary scorer assigns the observation an evidence value; `EvidenceGuidedMemory` keeps the complete trajectory for auditability but renders only the first, last, and top-*K* observations into the next context. Low-value exploration therefore stops competing with the passages most useful for identification, while no evidence is erased from persistent history. This harness is tailored to a retrieval task in which the main bottleneck is evidence salience rather than tool choice.

ABACUS is generated for a numerical analysis request that first compares regional averages and then ranks countries within the winning region. The harness deliberately moves aggregation out of language-model reasoning: `StructuredReActAction` invokes shell or Python, extracts a marked `RESULT_JSON` payload from standard output, repairs its syntax if necessary, and writes the values into

Appraiser — evidence decides what the model sees

Keep the full trajectory; render only the first, last, and highest-scoring evidence into the next context.

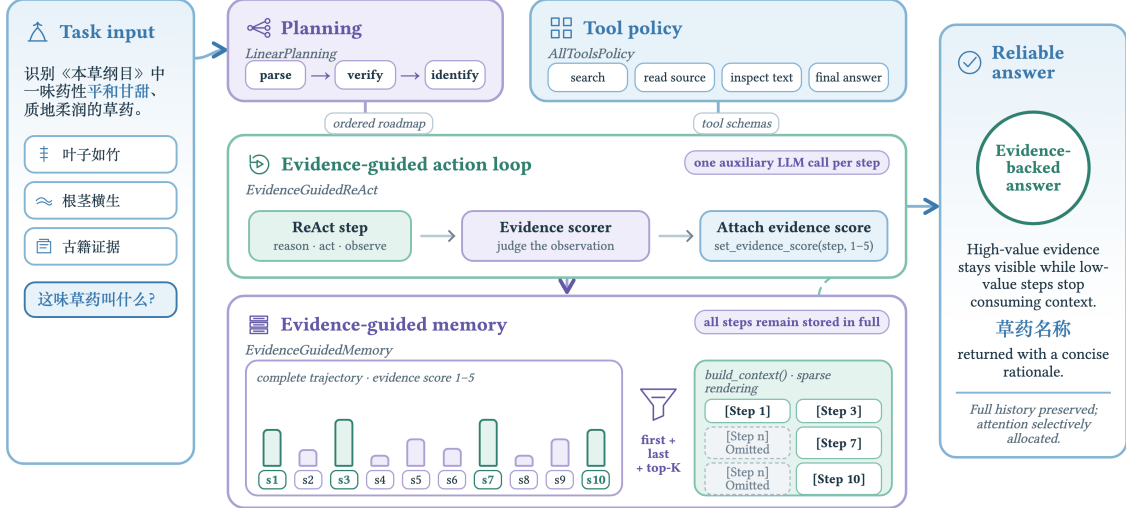


Figure A.5: Appraiser: evidence decides what the model sees. EvidenceGuidedReAct scores every step, while EvidenceGuidedMemory stores the full run but renders only the first, last, and top-K observations into build_context().

Abacus — from computation to reliable typed state

Shell-derived evidence is parsed once, preserved structurally, and reported without re-deriving the numbers.

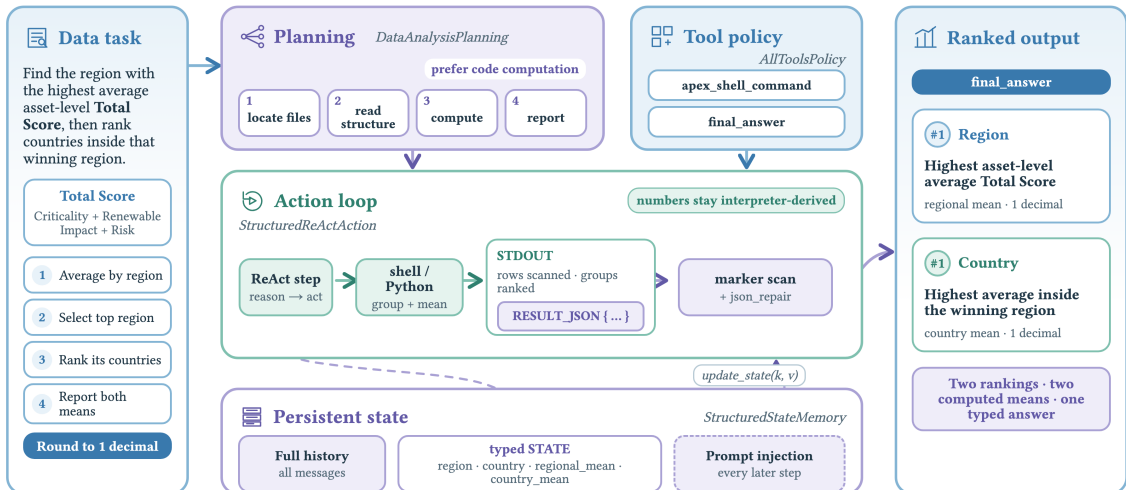


Figure A.6: Abacus: from computation to reliable typed state. StructuredReActAction executes shell or Python code, extracts a RESULT_JSON payload from standard output, and updates StructuredStateMemory without asking the model to re-derive computed values.

StructuredStateMemory. Region, country, regional mean, and country mean are then injected as typed state into every later step, so the final answer reports interpreter-derived quantities instead of re-deriving them from prose. Both cases preserve a full audit trail, but each renders the working view around its task’s dominant reliability requirement—evidence selection for one and numerical fidelity for the other.

JIT HARNESS ATLAS · 09

Player Piano – typed batch editing, verified file by file

A Python interpreter executes a structured plan; deterministic checks control every transition.

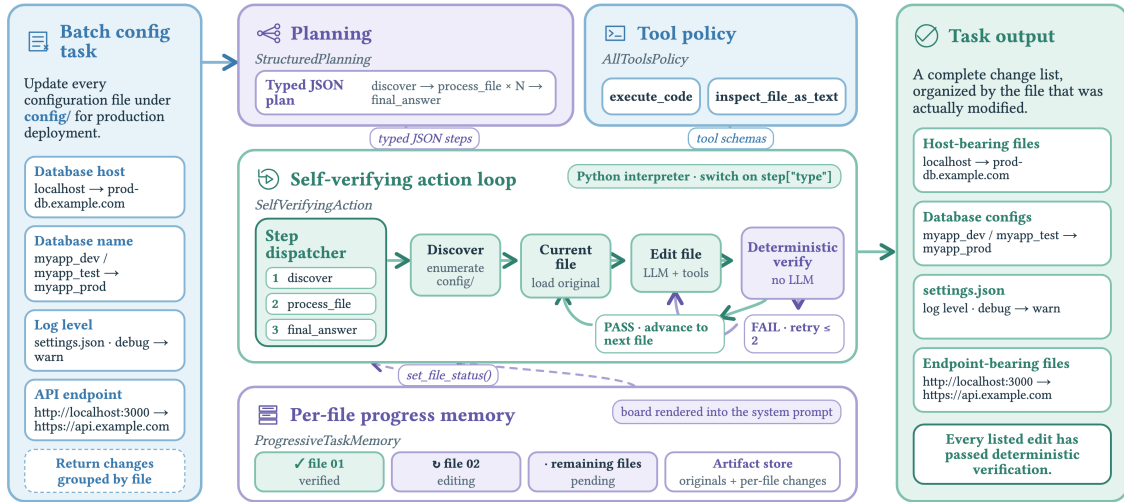


Figure A.7: Player Piano: typed batch editing, verified file by file. A Python dispatcher interprets *StructuredPlanning* steps, *SelfVerifyingAction* checks each edit without an LLM, and *ProgressiveTaskMemory* maintains a per-file board and artifact store.

Figures A.7 and A.8 specialize workspace execution at different scales by controlling when actions enter persistent history. *PLAYER PIANO* handles a batch migration of configuration files, including database, logging, and API-endpoint changes. *StructuredPlanning* emits typed *discover*, *process-file*, and *final-answer* steps that a Python dispatcher executes; after each edit, a deterministic non-LLM check decides whether to advance or retry. *ProgressiveTaskMemory* renders a per-file board into the system prompt and retains both originals and accepted changes, preventing a large batch from losing track of which files have actually passed verification.

MULLIGAN is generated for a shorter document task: extract a student identifier from a DOCX file and rename that same file. Its linear plan needs no hierarchy or context compression, so the harness retains full successful history and exposes only Word and file operations while hiding unrelated email, spreadsheet, and PDF tools. If an action fails, the loop appends the error locally and regenerates the same step for at most two attempts; only a successful action is committed before the plan advances. The two harnesses therefore share a concern for reliable workspace mutation but realize it differently: one uses typed batch progress and per-file verification, whereas the other uses narrow tool exposure and transactional step repair. This task-dependent difference is precisely the behavior sought from *JIT-Agent*: reliability mechanisms are generated to fit the structure and failure surface of the request rather than imposed as one universal scaffold.

Mulligan — repair the step, preserve the history

A failed action is regenerated in place; only successful repairs enter the transcript.

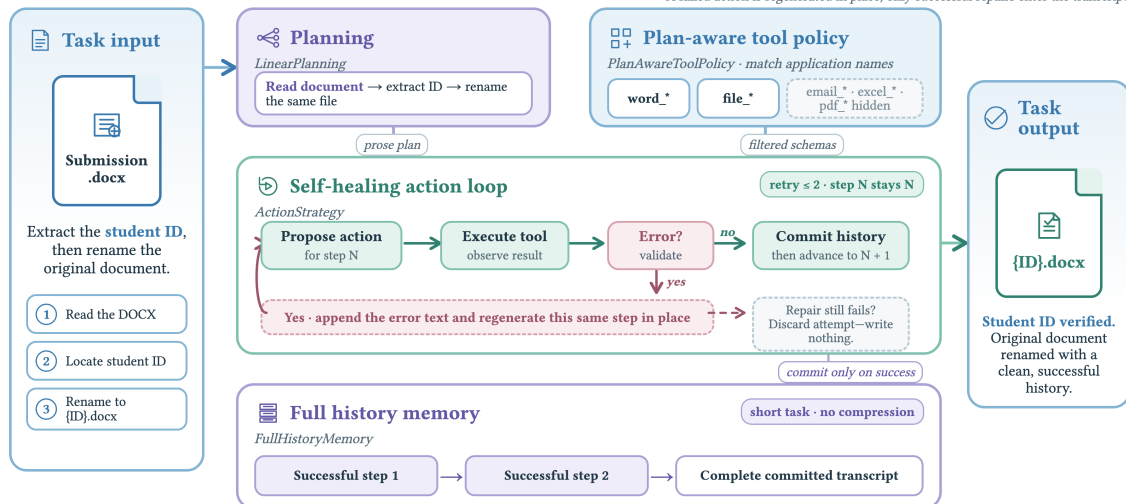


Figure A.8: Mulligan: repair the step, preserve the history. PlanAwareToolPolicy exposes only plan-relevant file and document tools; failed actions are regenerated at the same step, and FullHistoryMemory receives only the successful execution.